Embedded Software Architecture

System and Software Architecture Design

Version 0.3

Accepted Initialization

by

Nkeh P. Boh

Stevens Institute of Technology

nboh@stevens.edu

November 12, 2026

Project owner: Nkeh P. Boh
Organization: Stevens Institute of Technology
Contact: nboh@stevens.edu
Initial repository publication is authorized; see the publication record.

Document History (Appendix G)

Template Attribution

This software architecture design template was developed by **Darian Muresan, Ph.D.**, in conjunction with the AI agents **OpenAI Codex** and **Anthropic Claude Code**.

Contact: dmuresan@stevens.edu
darian@dmmd.net
darian@pictomic.com

Embedded Software Architecture

System and Software Architecture Design

Version 0.3
Accepted Initialization

Nkeh P. Boh
Stevens Institute of Technology
nboh@stevens.edu
November 12, 2026

This initialization draft establishes the architecture documentation workspace for Embedded Software Architecture, derived from ArchitectureDesignTemplate version 2.33. Nkeh P. Boh owns and approves the project. The scope is architecture documentation only. The Embedded Systems profile is Applicable. Technical facts, requirements, design choices, and product evidence have not been supplied; they remain explicit questions owned by Nkeh P. Boh in the owned-question register. The initialized documentation has been accepted and its publication authorized; technical baseline decisions and product release remain unresolved.

Table of Contents

Template Attribution iii

1 Introduction 1
- 1.1 Chapter Responsibility . . . 1
- 1.2 How to Use This Chapter . . . 1
- 1.3 Purpose . . . 1
- 1.4 Scope . . . 1
- 1.5 Audience . . . 2
- 1.6 Normative Language and Priority . . . 2
- 1.7 Document as Source of Truth . . . 2
- 1.8 Architecture as Maintained Input for AI Design . . . 2
- 1.9 Initializing a New Project from This Template . . . 3
- 1.10 Template Tailoring and Domain Profiles . . . 4
- 1.11 Using This Template with AI Agents . . . 4
- 1.12 V-Model Organization . . . 4

2 Development Plan 6
- 2.1 Chapter Responsibility . . . 6
- 2.2 Confirmed Project Decisions . . . 6
- 2.3 Domain Profile Selection . . . 6
- 2.4 Change Control and Execution . . . 7
- 2.5 Owned Questions . . . 7
- 2.6 Initialization Acceptance . . . 8

3 System Context 9
- 3.1 Chapter Responsibility . . . 9
- 3.2 How to Use This Chapter . . . 9
- 3.3 Project Status . . . 9

4 User Stories 10
- 4.1 Chapter Responsibility . . . 10
- 4.2 How to Use This Chapter . . . 10
- 4.3 Project Status . . . 10

5 Requirements **11**
5.1 Chapter Responsibility . . . 11
5.2 How to Use This Chapter . . . 11
5.3 Requirement Identifier Naming Convention . . . 11
5.4 Functional and Quality Requirements . . . 13
5.5 Requirement Status Model . . . 13
5.6 Requirement Category Catalog . . . 14
5.7 Domain Profile Impact on Requirements . . . 14
5.8 Creating a Project-Specific Category . . . 17
5.9 Review Rules . . . 18
5.10 Project Requirement Registers . . . 18
5.10.1 Business Requirements . . . 18
5.10.2 Functional Requirements . . . 18
5.10.3 Quality Requirements . . . 18
5.10.4 Interface Requirements . . . 18
5.10.5 Security Requirements . . . 18
5.10.6 Data Requirements . . . 18
5.10.7 Constraint Requirements . . . 19

6 Use Cases **20**
6.1 Chapter Responsibility . . . 20
6.2 How to Use This Chapter . . . 20
6.3 Project Status . . . 20

7 Architecture Overview **21**
7.1 Chapter Responsibility . . . 21
7.2 How to Use This Chapter . . . 21
7.3 Project Status . . . 21

8 User Interface Design **22**
8.1 Chapter Responsibility . . . 22
8.2 How to Use This Chapter . . . 22
8.3 Project Status . . . 22

9 Logical View **23**
9.1 Chapter Responsibility . . . 23
9.2 How to Use This Chapter . . . 23
9.3 Project Status . . . 23

10 Process View **24**
10.1 Chapter Responsibility . . . 24
10.2 How to Use This Chapter . . . 24
10.3 Project Status . . . 24

11 Development View **25**
11.1 Chapter Responsibility . . . 25
11.2 How to Use This Chapter . . . 25
11.3 Project Status . . . 25

12 Physical View **26**
12.1 Chapter Responsibility . . . 26
12.2 How to Use This Chapter . . . 26
12.3 Project Status . . . 26

13 Implementation **27**
13.1 Chapter Responsibility . . . 27
13.2 How to Use This Chapter . . . 27
13.3 Project Status . . . 27
13.4 Document Build . . . 27

14 Tests **28**
14.1 Chapter Responsibility . . . 28
14.2 How to Use This Chapter . . . 28
14.3 Project Status . . . 28

15 Traceability **29**
15.1 Chapter Responsibility . . . 29
15.2 How to Use This Chapter . . . 29
15.3 Project Status . . . 29

16 Verification **30**
16.1 Chapter Responsibility . . . 30
16.2 How to Use This Chapter . . . 30
16.3 Project Status . . . 30

17 Validation **31**
17.1 Chapter Responsibility . . . 31
17.2 How to Use This Chapter . . . 31
17.3 Project Status . . . 31

18 Release **32**
18.1 Chapter Responsibility . . . 32
18.2 How to Use This Chapter . . . 32
18.3 Initialization Acceptance and Publication . . . 32

A Document Authoring Guide **33**
A.1 Chapter Responsibility . . . 33
A.2 How to Use This Appendix . . . 33
A.3 Template Provenance . . . 33

A.4 AI-Assisted Authoring . . . 33
A.5 File Naming . . . 34
A.5.1 Document-Authoring Requirements . . . 34
A.6 One Responsibility per Chapter . . . 36
A.7 Adding a Chapter . . . 36
A.8 Diagram Authoring with TikZ . . . 37
A.9 Building the PDF . . . 37
A.9.1 Installing the Tools on macOS . . . 38
A.9.2 Installing the Tools on Windows . . . 38
A.9.3 Installing the Tools on Ubuntu or Debian Linux . . . 39
A.9.4 AI-Agent Installation . . . 39
A.9.5 Building on macOS and Linux . . . 40
A.9.6 Building with Windows PowerShell . . . 40
A.9.7 Manual Build Sequence . . . 40
A.9.8 Build Verification and Troubleshooting . . . 40
A.10 Authoring Rules . . . 41
A.11 Citations and Bibliography . . . 41
A.12 Selective Builds . . . 42
A.13 Project Naming Exceptions and Inherited Examples . . . 42

B Embedded Systems Tailoring Profile **43**
B.1 Chapter Responsibility . . . 43
B.2 How to Use This Appendix . . . 43
B.3 Applicability . . . 43
B.4 Core-Chapter Tailoring Map . . . 44
B.5 Hardware–Software Boundary and Allocation . . . 45
B.6 Embedded Requirement Categories . . . 45
B.7 Required Architecture Views . . . 46
B.8 Interfaces and Protocols . . . 47
B.9 Resource Budgets . . . 48
B.10 Safety, Reliability, and Fault Management . . . 48
B.11 Embedded Security . . . 49
B.12 Toolchain and Configuration Control . . . 49
B.13 Embedded Verification and Validation . . . 49
B.14 AI Tailoring Instructions . . . 49
B.15 Embedded-Profile Completion Checklist . . . 50

C AI Agent Usage Guide **52**
C.1 Chapter Responsibility . . . 52
C.2 How to Use This Appendix . . . 52
C.3 Governing Principles . . . 52
C.4 AI Quick Start . . . 53
C.5 Repository AI Bootstrap . . . 54
C.6 Guide Activity . . . 56

C.7 AI Skills Register . . . 56
C.8 Task-Specific Context Packs . . . 60
C.8.1 Planning and Estimation Context . . . 60
C.8.2 Domain-Profile Context . . . 60
C.8.3 User-Story Context . . . 60
C.8.4 Requirements Context . . . 61
C.8.5 Use-Case Context . . . 61
C.8.6 Architecture Context . . . 61
C.8.7 Testing and Evidence Context . . . 61
C.8.8 Release Context . . . 61
C.8.9 Document-Authoring Context . . . 62
C.9 Mandatory Consistency Rules . . . 62
C.10 Skill-Specific Steps and Acceptance . . . 63
C.10.1 Estimate AI Work . . . 63
C.10.2 Initialize a Project . . . 64
C.10.3 Tailor an Embedded System . . . 64
C.10.4 Create Repository AI Bootstrap Files . . . 64
C.10.5 Develop User Stories . . . 65
C.10.6 Add or Change a Requirement . . . 65
C.10.7 Add or Change a Use Case . . . 65
C.10.8 Develop or Change Architecture . . . 66
C.10.9 Add Tests or Evidence . . . 66
C.10.10 Perform Verification . . . 66
C.10.11 Perform Validation . . . 67
C.10.12 Review and Prepare a Release . . . 67
C.10.13 Review Consistency . . . 67
C.10.14 Compile or Troubleshoot . . . 68
C.11 Example AI Requests and Prompts . . . 68
C.11.1 Initialize the Architecture Book . . . 68
C.11.2 Tailor an Embedded System . . . 69
C.11.3 Create Repository AI Bootstrap Files . . . 69
C.11.4 Develop Requirements . . . 69
C.11.5 Develop Use Cases . . . 69
C.11.6 Develop an Architecture Decision . . . 70
C.11.7 Develop Tests and Traceability . . . 70
C.11.8 Review Release Readiness . . . 70
C.12 Human Decisions and Information Boundaries . . . 70
C.13 Completion Checklist . . . 71

D AI-Assisted Execution Workflow **73**
D.1 Chapter Responsibility . . . 73
D.2 Purpose and Scope . . . 73
D.3 Starting the Workflow in an AI Chat . . . 74
D.3.1 Start Any Selected AI Skill . . . 75

D.3.2 Develop an Approved Change . . . 75
D.3.3 Prepare a Requirements Package . . . 76
D.3.4 Other AI Tools . . . 76
D.4 Plan the Selected Tasks . . . 77
D.5 Choose and Invoke Agents . . . 77
D.6 Responsibilities and Handoffs . . . 78
D.7 Develop, Integrate, and Verify . . . 79
D.8 Validate with Stakeholders . . . 79
D.9 Project Tailoring Checklist . . . 80

E AI Prompt Log **83**
E.1 Chapter Responsibility . . . 83
E.2 How to Use This Appendix . . . 83
E.3 Initialization Interaction . . . 83
E.4 Owner Identity Correction . . . 83
E.5 Publication Authorization . . . 84
E.6 Requirement Scope or Family Extension Record . . . 84

F Test Evidence **85**
F.1 Chapter Responsibility . . . 85
F.2 How to Use This Appendix . . . 85
F.3 Project Status . . . 85

G Document History **86**
G.1 Chapter Responsibility . . . 86
G.2 Project History . . . 86
G.3 Template Provenance . . . 86

Glossary **87**

Acronyms **88**

Bibliography **89**

Index **90**

List of Tables

2.1 Project Domain Profile Decisions 6
2.2 Owned Initialization Questions 7

5.1 Requirement Identifier Segments and Family Codes 12
5.2 Requirement Status Definitions 13
5.3 Requirement Families, Subtypes, and Quality Attributes 15

A.1 Document-Authoring Requirements 34

B.1 Embedded-Systems Core-Chapter Tailoring Map 44
B.2 Embedded-System Requirement Categories 45
B.3 Candidate Embedded-System Architecture Views 47
B.4 Generic Embedded Resource Budget 48

C.1 Repository AI Bootstrap Files 55
C.2 AI Skills Register 58
C.3 AI-Assisted Change Completion Checklist 71

List of Figures

1.1 Maintained source in compiler and AI workflows, retained from template 2.33 [1]. . . . 3
1.2 Human and AI-assisted architecture feedback, retained from template 2.33 [1]. 3
1.3 V-model relationships, agile iteration, and operational feedback, retained from template 2.33 [1]. . . . 5

C.1 AI agent activity from repository instructions through the generic skill, AIS selection, Context, Steps, Acceptance, and execution. . . . 57

D.1 AI-assisted development and validation handoffs. . . . 81
D.2 Activity connecting Appendix C skill definitions and Acceptance to Appendix D execution. The agent-count decision belongs to Appendix D for every selected task. . . . 82

Chapter 1

Introduction

1.1 Chapter Responsibility

Establish the purpose, scope, audience, and conventions of the engineering record.
Expected result: A shared understanding of how to read and use the architecture.

1.2 How to Use This Chapter

State the document purpose, scope, audience, conventions, and relationship to other project artifacts. Replace bracketed text with project-specific content. Keep binding obligations in the Requirements chapter.

1.3 Purpose

This document is the authoritative engineering specification for Embedded Software Architecture. It records why the system is needed, what it shall do, how it is structured, how it will be tested, and what evidence is required before release. It is readable by people and structured for use by engineering and artificial intelligence (AI)-assisted development tools.

Central lesson

A vague request produces unpredictable software. A precise, traceable specification allows engineers to generate, review, test, and, when necessary, regenerate a conforming implementation.

1.4 Scope

The authorized work is architecture documentation only. It includes initializing this architecture book, retaining editable TikZ sources and trace links, recording supported facts and owned questions, building the complete PDF, and preparing the candidate for owner review.

Product implementation, product verification, stakeholder validation, and release are not established by this initialization. The system purpose and product boundary remain Q-02.

1.5 Audience

Nkeh P. Boh is the confirmed document owner and approver. Additional stakeholders and delegated roles remain Q-01.

1.6 Normative Language and Priority

Use **shall** for every binding requirement. Do not use **should** as guidance in normative statements. Assign each requirement one priority: **Must**, **Should**, **Could**, or **Would**. Priority expresses delivery importance; it does not change whether a statement containing **shall** is binding for its assigned implementation version.

1.7 Document as Source of Truth

Requirements alone do not define a reproducible implementation. The complete source of truth consists of stakeholder needs, user stories, requirements, use cases, architecture and interface decisions, implementation constraints, tests, verification evidence, validation evidence, and release criteria. Generated code is an implementation artifact governed by this specification.

1.8 Architecture as Maintained Input for AI Design

Figure 1.1 distinguishes two workflows. A compiler regenerates lower-level output from reviewed C/C++ source and its build configuration. An AI request is often incomplete: it may omit interface assumptions, error behavior, or earlier decisions. Generated code therefore requires review, tests, and version control. Repeating a prompt does not reproduce an approved implementation by itself.

The goal of this template is to make the approved architecture the maintained engineering input to AI-assisted design and implementation. Its linked needs, requirements, interfaces, responsibilities, constraints, and rationale give a bounded task its context. The task request identifies the affected baseline and change; the architecture remains the authoritative record of design intent. This analogy does not make an AI agent a deterministic compiler. Engineers must review, verify, and version its code and tests, and resolve missing acceptance criteria before claiming conformance.

Figure 1.2 shows how implementation and test findings can improve the architecture. An engineer may edit the affected views directly, or an AI agent may prepare a proposed update. In either route, authorized reviewers assess the evidence and approve changes before they enter the baseline. A finding may affect requirements, interfaces, allocations, and

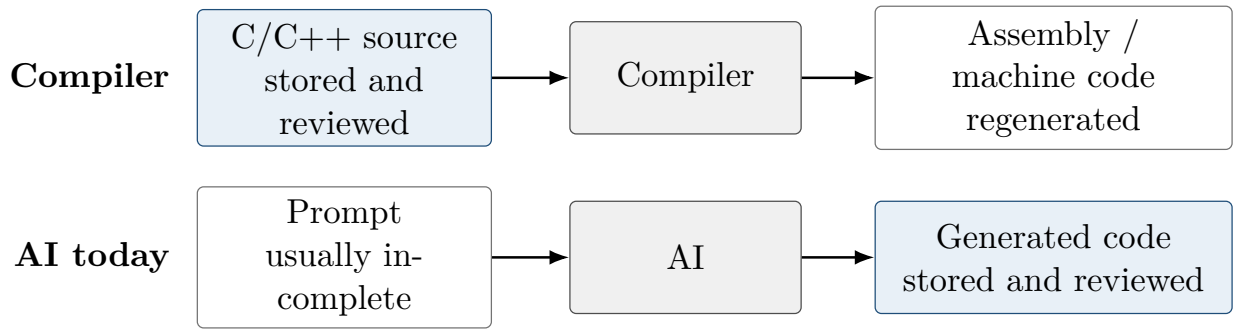


Figure 1.1: Maintained source in compiler and AI workflows, retained from template 2.33 [1].

planned checks together; update those links before using the revised baseline for the next implementation task. Chapter C and Chapter D define the task and execution controls.

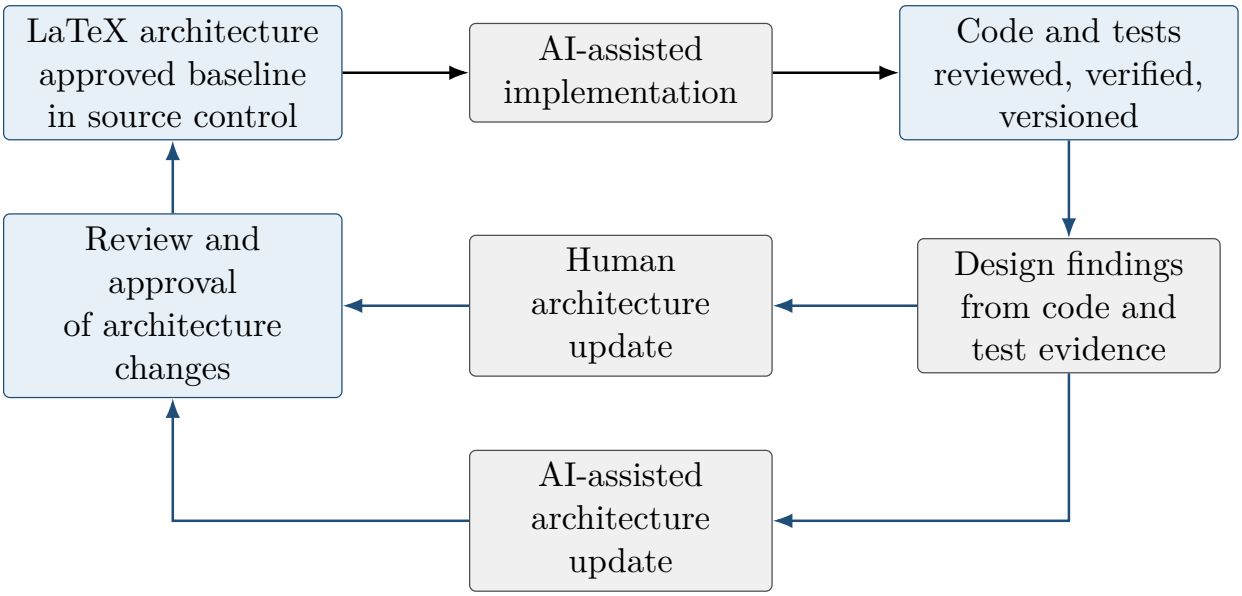


Figure 1.2: Human and AI-assisted architecture feedback, retained from template 2.33 [1].

1.9 Initializing a New Project from This Template

Create a project-specific architecture book from a copy of this template; keep the master template generic. Use AIS-02: Initialize a Project for the authoritative Context, Steps, and Acceptance, including confirmation of project information, review of supplied evidence, and replacement of supported placeholders.

To start initialization with an AI agent, use the request in Section C.11.1, Initialize the Architecture Book. Execute the task using Chapter D.

1.10 Template Tailoring and Domain Profiles

The core chapters form a generic software-architecture template. A domain tailoring profile augments those chapters with concerns, artifacts, diagrams, and review criteria for a particular class of system. It does not replace a core chapter or create a second authoritative location for project facts. Requirements, interfaces, architecture decisions, tests, and evidence remain in their normal core chapters and link to one another through their stable IDs.

Select applicable profiles during project initialization. More than one profile may apply. Record each applicability decision and rationale in Chapter 2, retain applicable profile appendices as authoritative tailoring guidance, and exclude an inapplicable profile from an instantiated project only after recording the decision. If profile guidance conflicts, identify the conflict and obtain an authorized human decision before editing controlled project content.

The authorized applicability decision is recorded once in the project profile register.

When the Embedded Systems profile is Applicable or TBD, read Chapter B during initialization and for every task affected by hardware, firmware, real-time behavior, constrained resources, physical interfaces, safety, security, manufacturing, or field service.

1.11 Using This Template with AI Agents

Use Chapter C to select AI Skills and read their linked Context, Steps, and Acceptance. Use Chapter D for execution with one or multiple agents. To start a task, use Start Any Selected AIS; the task-specific example requests provide additional starting prompts. The guide activity diagram shows the reading route from repository bootstrap files to task selection and execution.

1.12 V-Model Organization

Figure 1.3 connects development decisions with evidence. Stakeholder needs motivate requirements, which guide architecture, detailed design, and implementation. Unit tests address detailed behavior; integration tests address interactions; system verification assesses specified obligations; and stakeholder validation assesses intended use. These correspondences help identify the evidence needed for each design decision.

The model includes iteration. Validation findings can return to stakeholder needs during development. After deployment, operational findings and change requests can trigger another cycle. A changed baseline requires review of its requirements, design links, and affected checks. This organization is compatible with the system and software life-cycle process frameworks in [2, 3]; those standards do not prescribe one life-cycle model.

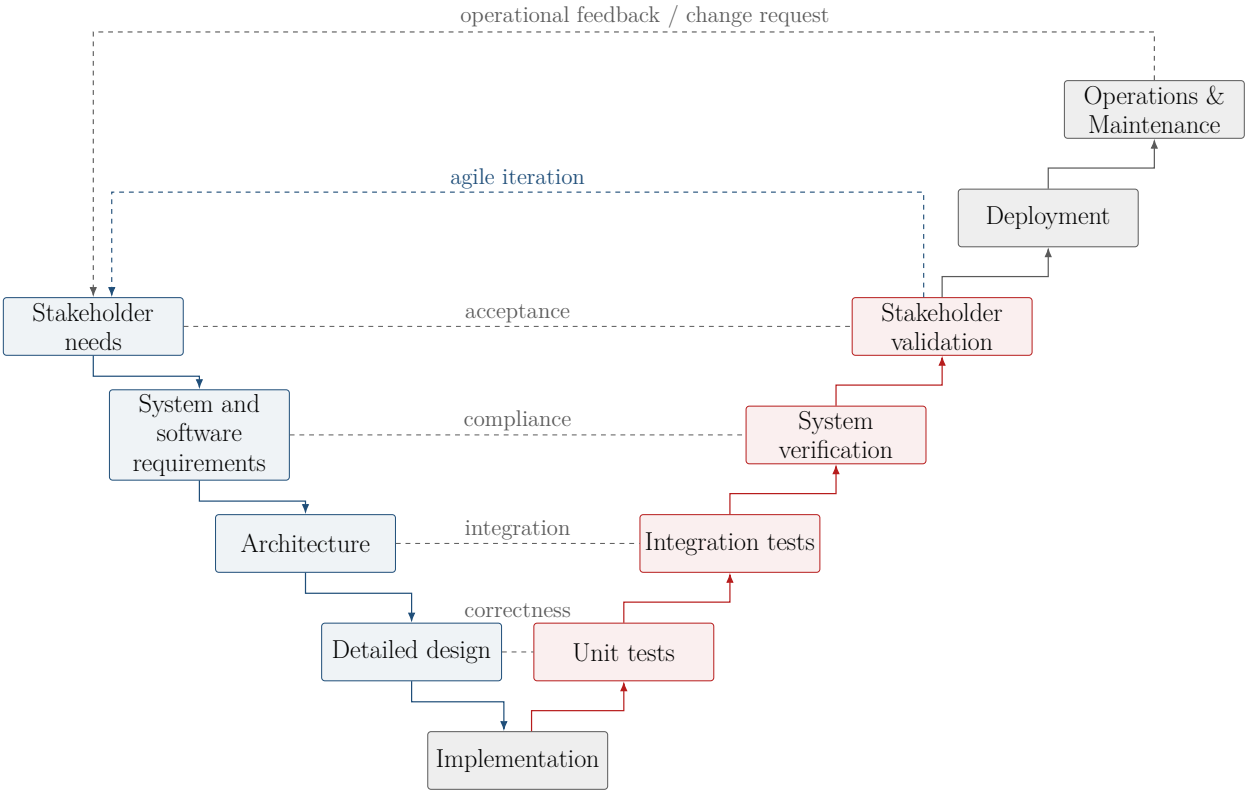


Figure 1.3: V-model relationships, agile iteration, and operational feedback, retained from template 2.33 [1].

Chapter 2

Development Plan

2.1 Chapter Responsibility

Define governance, initialization boundaries, and ownership of unresolved decisions. **Expected result:** A reviewable documentation candidate with explicit decision gates.

2.2 Confirmed Project Decisions

Nkeh P. Boh confirmed the name Embedded Software Architecture, namespace `ftdi2`, architecture-documentation-only scope, and their role as owner and approver. Unknown technical details are questions owned by Nkeh P. Boh. These decisions were supplied in this initialization chat on November 12, 2026. The owner subsequently supplied the identity below.

Owner and approver Nkeh P. Boh

Organization Stevens Institute of Technology

Email nboh@stevens.edu

Publication rights, other stakeholders, and delegated roles remain Q-01.

2.3 Domain Profile Selection

Table 2.1: Project Domain Profile Decisions

Profile	Status	Rationale, Owner, and Date
Embedded Systems	Applicable	Explicitly selected by Nkeh P. Boh, owner and approver, on November 12, 2026. Technical platform and hardware basis remain Q-03.

Embedded Systems is the only profile supplied by template 2.33. Any additional assurance or profile needs remain Q-09. The retained profile guides assessment; it does not establish hardware facts or product obligations.

2.4 Change Control and Execution

Codex is the AI tool used for this initialization. One agent coordinates, drafts, builds, and performs self-review; no independent review is claimed. Nkeh P. Boh owns and approves scope, architecture, evidence, and acceptance. Technical decisions remain blocked until supported and approved; independent documentation preparation may continue.

Nkeh P. Boh accepted the initialized project and authorized initial publication on November 12, 2026. The authorized target and package are recorded in the publication record. The destination already contained Git on main at commit `2673ab5`; retain that history, commit the initialized sources on main, and push to the authorized repository. Every later change uses a branch and pull request. The local owner-correction branch belongs to the initialization preparation; its uncommitted sources are included in this initial main publication.

2.5 Owned Questions

Questions Q-01 through Q-10 remain Open and owned by Nkeh P. Boh. Q-11 is Resolved by the explicit owner decision linked in its row. Answers require supporting evidence or an explicit decision. No due dates were supplied. Questions are not product requirements and do not claim completed traceability.

Table 2.2: Owned Initialization Questions

ID	Topic	Question and Authoritative Destination
Q-01	Identity and governance	What publication rights, additional named stakeholders, and delegated roles apply? Destination: Chapter 2.
Q-02	Purpose and scope	What embedded system is being documented, why is it needed, who uses it, and what capabilities, exclusions, operating environment, and external systems define its boundary? Destination: Chapter 3.
Q-03	Platform and allocation	What platform class, processors, boards, hardware revisions, sensors, actuators, hardware–software allocations, firmware layers, dependencies, and deployment variants apply? Destination: Chapter 7.
Q-04	Interface contracts	What physical, software, user, service, and communication interfaces apply? Supply controlled pinouts, electrical limits, protocols, formats, rates, units, error behavior, and interface owners. Destination: Chapter 3.
Q-05	Timing and resources	What deadlines, jitter, scheduling, interrupts, memory, CPU, storage, bandwidth, power, thermal limits, margins, and measurement conditions apply? Destination: Chapter 5.

ID	Topic	Question and Authoritative Destination
Q-06	Toolchain and configuration	What boot chain, BSP, HAL, RTOS/OS, SDK, compiler, linker, libraries, generated inputs, tool versions, build variants, and hardware compatibility are controlled? Destination: Chapter 13.
Q-07	Safety and security	What hazards, safe states, fault responses, reliability targets, threats, trust boundaries, provisioning, key custody, debug access, and safety/security decision authorities apply? Destination: Chapter 5.
Q-08	Lifecycle and environment	What environment, calibration, diagnostics, manufacturing, updates, rollback, recovery, maintenance, service, and component-obsolescence constraints apply? Destination: Chapter 5.
Q-09	Standards and assurance	What standards, regulatory or certification obligations, independent reviews, lifecycle gates, assurance evidence, and additional domain profiles are needed? Destination: Chapter 2.
Q-10	Needs and evidence	What controlled stakeholder needs, requirements, interface documents, schematics, datasheets, analyses, test environments, procedures, results, and validation criteria are available? Destination: Chapter 14.
Q-11	Acceptance and publication	Resolved November 12, 2026: owner acceptance and publication authorization received for the existing repository. See the authorized publication record. Destination: Chapter 18.

2.6 Initialization Acceptance

Acceptance requires supported metadata and profile decisions, visible owned questions, a complete document build, no undefined citations or references, unique labels, preserved TikZ sources and link targets, and visual inspection of the candidate PDF. Build evidence belongs in the namespaced review report. Product requirements, architecture decisions, tests, verification, validation, and release remain unpopulated until supporting facts are supplied. A successful document build does not close technical gaps. Owner acceptance was subsequently received as recorded in the publication record.

Chapter 3

System Context

3.1 Chapter Responsibility

Define the system boundary and its relationship to the surrounding environment.
Expected result: An agreed boundary for requirements and design decisions.

3.2 How to Use This Chapter

Define the problem, stakeholders, system boundary, external actors and systems, assumptions, dependencies, and exclusions. Add a context diagram when it clarifies interactions.

3.3 Project Status

The system purpose, operational boundary, actors, and external systems are unresolved. See Q-02, owned by Nkeh P. Boh.

Chapter 4

User Stories

4.1 Chapter Responsibility

Capture who needs a result and why that result has value.
Expected result: A reviewable statement of stakeholder value and acceptance intent.

4.2 How to Use This Chapter

Capture stakeholder goals in accessible language. Stories express value and acceptance intent; precise obligations belong in Requirements. Give each story a stable ID, a priority, and links to one or more requirements that implement it.

4.3 Project Status

No project user stories have been supplied. See Q-02, owned by Nkeh P. Boh.

Chapter 5

Requirements

5.1 Chapter Responsibility

Specify the obligations the system must satisfy in a verifiable form.
Expected result: A controlled set of obligations that can be implemented and assessed.

5.2 How to Use This Chapter

State singular, necessary, unambiguous, feasible, and verifiable obligations. Every requirement shall have a stable ID, one priority, the version in which it was implemented, at least one mapped use case, and at least one mapped test case. Enter ∅ in **In Version** when no implementation version has been assigned. Preserve IDs when wording or version assignments change.

Select the requirement categories that make the specification easier to review and govern. Keep each requirement in one primary register and use traceability links instead of copying the same requirement into multiple categories.

5.3 Requirement Identifier Naming Convention

Requirement identifiers use the following grammar:

`REQ-[<scope>-]<family>-<sequence>`

The optional scope qualifier identifies a project-defined subsystem, product, content type, or other governed area. Omit it for project-wide requirements. The sequence is a three-digit, zero-padded number that is unique within the exact scope and family combination. For example, `REQ-F-004` is the fourth project-wide functional requirement, while `REQ-API-Q-003` is the third quality requirement in a project-defined `API` scope.

Table 5.1: Requirement Identifier Segments and Family Codes

Code	Meaning	Use
`REQ`	Requirement	Identifies the artifact as a requirement. Every requirement ID begins with this segment.
`<scope>`	Project-defined scope	Optional uppercase qualifier for a governed subsystem, product, content type, or other area. Define every scope code before use; absence denotes project-wide scope.
`B`	Business	States the stakeholder outcome, mission, commercial purpose, policy goal, or business result the system shall enable.
`F`	Functional	States observable behavior, service, workflow, calculation, state change, or capability the system shall provide.
`Q`	Quality	States a measurable quality attribute or non-functional expectation, including accessibility, performance, reliability, resilience, usability, maintainability, or compatibility.
`I`	Interface	States a user, software, API, hardware, communication, protocol, or external-system boundary contract, including integration behavior at that boundary.
`S`	Security	States security, privacy, consent, authorization, auditing, secret-handling, or protected-data obligations maintained in a specialized security register.
`D`	Data	States data definition, authority, provenance, quality, integrity, ownership, lifecycle, retention, or source-of-truth obligations.
`C`	Constraint	States a binding implementation, platform, technology, format, storage, source-control, standards, compatibility, schedule, budget, or governance constraint.
`###`	Sequence	Three-digit, zero-padded stable sequence within the exact scope and family; assigned monotonically and never reused.

Classify a requirement by its primary obligation and verification concern. A requirement may support several concerns, but its identifier contains only one family code. Represent cross-cutting relationships through traceability rather than a compound family code.

An AI agent shall reuse an existing scope and family whenever their definitions fit. If no defined family accurately represents the requirement, the agent shall not silently invent or use a new code. It shall propose a unique uppercase code, full family name, decision boundary, rationale, and representative examples; check the complete requirements register and category catalog for semantic overlap and identifier collision; and obtain authorized-human approval. After approval, the agent shall add the family to Table 5.1, update the category catalog and AI guidance, and create or update the applicable register before assigning an identifier from it. The same approval and documentation rule applies to a new scope qualifier. Extending the convention shall not rename, renumber, or reuse an existing

identifier.

5.4 Functional and Quality Requirements

Functional and quality requirements answer different questions and shall not be used interchangeably.

Central Distinction

A **functional requirement** states what behavior, service, calculation, state transition, or externally observable result the system shall provide. A **quality requirement** states how well that behavior or the system as a whole shall perform under stated conditions. Quality requirements require a measurable threshold, workload, environment, or other evaluation basis.

For example, "The system shall display the current request status" is functional. "The system shall display the current request status within two seconds under the reference workload" is a quality requirement. Avoid vague quality statements such as "the system shall be fast, secure, reliable, or easy to use." Replace them with measurable conditions and success criteria.

Quality requirements are also called quality-attribute or non-functional requirements. The term *quality requirement* is preferred here because these obligations describe behavior that shall be designed, implemented, and verified rather than an absence of function.

5.5 Requirement Status Model

Priority and status are independent. Priority records the requirement's importance or commitment; status records its progress through the delivery lifecycle. A status change shall not erase or replace the assigned priority.

Table 5.2: Requirement Status Definitions

Status	Meaning	In Version Rule
Open	The requirement is active, but implementation has not started. Deferred work remains Open until work begins or the requirement is retired.	Enter ∅.
In Progress	Implementation or integration work has started, but the requirement does not yet satisfy the definition of Done.	Enter ∅.

Status	Meaning	In Version Rule
Done	The requirement is implemented; mapped tests pass; verification is complete; traceability is current; and the completed behavior is included in a controlled version.	Enter the first controlled version containing the completed requirement.
Retired	The requirement is no longer active because it was superseded, made obsolete, rejected, or withdrawn. Retain its stable ID, original text, and traceability; record the rationale and any replacement in document history.	Retain the original implementation version if it was previously Done; otherwise enter ∅.

Only **Done** asserts completion. Do not remove a retired requirement or reuse its ID. A requirement may move from **Retired** only through an approved change that restores its active status and records the decision in document history.

5.6 Requirement Category Catalog

The following catalog distinguishes broad requirement families from their subtypes. In particular, performance, capacity, scalability, reliability, availability, resilience, recoverability, usability, accessibility, maintainability, supportability, testability, portability, compatibility, interoperability, security, privacy, safety, sustainability, and observability are quality attributes. A requirement concerning one of these attributes is a quality requirement when it specifies how well the system shall operate under stated conditions.

A project may organize a specialized quality attribute, such as security, privacy, or safety, in a separate register when it has distinct owners, review criteria, evidence, or assurance obligations. The separate register is a management view; it does not change the requirement's classification as a quality requirement. Likewise, interface, data, operational, and other families may contain both functional obligations and associated quality obligations. Use the primary purpose of each requirement to select its register and use links to represent cross-cutting concerns.

The catalog is intentionally broad. A project may use a family as a top-level register, create registers for selected subtypes, combine closely related groups, or omit groups that do not apply. Category choice shall not weaken the traceability or verification rules for individual requirements.

5.7 Domain Profile Impact on Requirements

Every Applicable domain profile shall be evaluated when developing or changing requirements. Each applicable profile concern shall result in one or more project requirements,

a reference to an existing requirement, or a documented Not Applicable decision with rationale approved by an authorized human. Record the profile applicability decision in the Development Plan and retain project-specific obligations only in the applicable requirement registers. The profile itself is a tailoring guide, not a duplicate requirements specification. See Section 1.10 and the applicable profile appendix.

Table 5.3: Requirement Families, Subtypes, and Quality Attributes

Requirement Family	**Subtype or Quality Attribute**	**Typical Scope**
Business	Outcomes and policy	Organizational outcomes, value, policy goals, measurable benefits, cost targets, and mission objectives.
Stakeholder and user	Stakeholder need	Capabilities, outcomes, and constraints needed by a stakeholder group or external party.
Stakeholder and user	User need	User goals and externally visible outcomes expressed independently of a particular implementation.
Functional	System behavior	Services, calculations, decisions, state changes, workflows, and observable system responses.
Quality	General quality	Measurable characteristics that state how well a function or the system shall operate under stated conditions.
Quality	Performance	Response time, latency, throughput, processing time, and resource consumption.
Quality	Capacity	Supported data volume, transaction volume, concurrent users, devices, or retained records.
Quality	Scalability	Behavior as workload, data, users, or deployed instances increase or decrease.
Quality	Reliability	Probability or frequency of failure-free operation under stated conditions.
Quality	Availability	Required service availability, planned downtime, and permitted outage duration.
Quality	Resilience and fault tolerance	Continued or degraded operation when components, dependencies, or communication paths fail.
Quality	Recoverability and continuity	Recovery time, recovery point, restoration, backup, failover, and disaster-continuity objectives.
Quality	Usability	Task effectiveness, efficiency, learnability, error prevention, and user satisfaction.
Quality	Accessibility	Conformance and measurable support for users with visual, auditory, motor, speech, or cognitive disabilities.

Continued on next page

Requirement Families, Subtypes, and Quality Attributes (continued)

Requirement Family	Subtype or Quality Attribute	Typical Scope
Quality	Maintainability	Time, effort, modularity, analyzability, and change characteristics required for system modification.
Quality	Supportability	Diagnostic access, service procedures, field replacement, support hours, and support-team capabilities.
Quality	Testability	Controllability, observability, repeatability, test interfaces, and evidence needed to verify behavior.
Quality	Portability	Ability to install, move, or operate the system across specified environments or platforms.
Quality	Compatibility	Coexistence and correct operation with specified versions, devices, browsers, operating systems, or products.
Quality	Interoperability	Coordinated behavior, protocols, semantics, orchestration, and exchanges with external systems.
Quality	Security	Authentication, authorization, confidentiality, integrity, cryptography, auditing, and threat resistance.
Quality	Privacy	Collection, consent, purpose, minimization, access, disclosure, retention, deletion, and rights for personal data.
Quality	Safety	Hazard prevention, risk reduction, safe state, interlocks, alarms, and unacceptable-loss controls.
Quality	Sustainability	Energy use, resource efficiency, carbon reporting, hardware life, repair, reuse, and disposal objectives.
Quality	Observability	Logs, metrics, traces, health signals, diagnostic context, and measurable diagnostic coverage.
Interface	User interface	Human-system interaction, presentation, navigation, input, feedback, and interaction contracts.
Interface	Software and API	Operations, messages, schemas, errors, protocols, sequencing, and software boundary contracts.
Interface	Hardware and communication	Connectors, signals, electrical characteristics, device protocols, networks, and physical boundaries.
Data and information	Data definition and quality	Data definitions, schemas, provenance, accuracy, completeness, integrity, ownership, and permitted transformations.
Data and information	Lifecycle and governance	Creation, access, retention, archival, legal hold, disposition, deletion, and proof of destruction.

Continued on next page

Requirement Families, Subtypes, and Quality Attributes (continued)

Requirement Family	Subtype or Quality Attribute	Typical Scope
Operational	Operation and administration	Startup, shutdown, scheduling, configuration, provisioning, monitoring, backup, recovery, and incident response.
Operational	Deployment and platform	Approved platforms, topology, packaging, installation, configuration, upgrade, and runtime environment.
Operational	Infrastructure and environment	Network, compute, storage, facility, power, temperature, physical location, and environmental limits.
Transition	Migration and rollout	Data conversion, coexistence, rollout, cutover, rollback, legacy integration, and decommissioning.
Compliance and legal	Regulatory and standards	Obligations imposed by regulations, standards, certifications, policies, or assurance frameworks.
Compliance and legal	Legal and contractual	Licenses, warranties, intellectual-property terms, service agreements, procurement terms, and contractual duties.
Compliance and legal	Reporting and audit	Reports, evidence, audit trails, review access, content, timing, and record accountability.
Product support	Documentation	Required user, operator, administrator, developer, API, maintenance, and compliance documentation.
Product support	Training	Required curricula, exercises, qualifications, delivery modes, and demonstrated learner outcomes.
Commercial	Licensing and entitlement	Product editions, entitlement, metering, third-party components, subscriptions, and licensing restrictions.
AI and machine learning	Model and data controls	Model performance, data provenance, bias evaluation, explainability, human oversight, drift, and retraining controls.
Physical and hardware	Physical characteristics	Dimensions, mass, materials, connectors, tolerances, environmental ratings, and hardware behavior.
Constraint	Solution constraint	Mandated technologies, designs, budgets, schedules, standards, dependencies, or solution boundaries.

5.8 Creating a Project-Specific Category

Create an additional category when it gives requirements a distinct meaning, owner, review method, or assurance process. Define the category's scope and a unique family code

using Section 5.3; duplicate one of the category-specific registers below; keep the standard columns; add the category to the category catalog; and update use cases, tests, traceability, verification, validation, document history, and the index as applicable. Document whether the new category is top-level or a subtype of an existing category.

Do not create a category solely to avoid resolving an ambiguous requirement. If one statement contains multiple independently verifiable obligations, split it into singular requirements and assign each one a primary category.

5.9 Review Rules

Reject a requirement row if its priority, status, use-case mapping, or test-case mapping is empty. The **Status** field shall contain Open, In Progress, Done, or Retired. The **In Version** field shall follow the status definitions. Confirm that reciprocal mappings appear in the user-story, use-case, test, traceability, and verification registers.

5.10 Project Requirement Registers

No project product requirements have been supplied. Template service-request examples were not adopted. No project requirement has a Done status or an implementation version. The embedded concern assessment remains open under Q-05, Q-07, Q-08, and Q-09.

5.10.1 Business Requirements

No project rows; awaiting owner-supplied evidence and approval.

5.10.2 Functional Requirements

No project rows; awaiting owner-supplied evidence and approval.

5.10.3 Quality Requirements

No project rows; awaiting owner-supplied evidence and approval.

5.10.4 Interface Requirements

No project rows; awaiting owner-supplied evidence and approval.

5.10.5 Security Requirements

No project rows; awaiting owner-supplied evidence and approval.

5.10.6 Data Requirements

No project rows; awaiting owner-supplied evidence and approval.

5.10.7 Constraint Requirements

No project rows; awaiting owner-supplied evidence and approval.

Chapter 6

Use Cases

6.1 Chapter Responsibility

Describe how actors interact with the system to achieve their goals.
Expected result: Complete interaction scenarios tied to requirements and tests.

6.2 How to Use This Chapter

Describe goal-oriented interactions at the system boundary. Give every use case a stable ID, its own table, and at least one reciprocal requirement mapping. Include preconditions, trigger, main flow, alternate or exception flows, and postconditions. Maintain the canonical relationships among stories, requirements, use cases, and tests in Chapter 15.

6.3 Project Status

No project use cases or system-boundary diagram are asserted. See Q-02, owned by Nkeh P. Boh.

Chapter 7

Architecture Overview

7.1 Chapter Responsibility

Explain the major design choices and why they address the design drivers.
Expected result: A defensible design direction with explicit trade-offs.

7.2 How to Use This Chapter

Explain the forces that shape the design, the major decisions, alternatives considered, and consequences. Use the 4+1 views only when they help readers answer different architectural questions.

7.3 Project Status

No project architecture decision has been proposed or approved. See Q-03, owned by Nkeh P. Boh.

Chapter 8

User Interface Design

8.1 Chapter Responsibility

Define how people interact with the system and understand its state.
Expected result: An interaction design that supports stakeholder tasks and acceptance.

8.2 How to Use This Chapter

Describe personas, primary screens or workflows, interaction rules, feedback, accessibility, and essential states. Include wireframes or prototypes where they resolve meaningful ambiguity.

8.3 Project Status

User and service interfaces are unresolved; no user interface is assumed. See Q-04, owned by Nkeh P. Boh.

Chapter 9

Logical View

9.1 Chapter Responsibility

Define the concepts, classes, interfaces, and responsibilities of the design.
Expected result: A structural model with explicit responsibility and ownership.

9.2 How to Use This Chapter

Model the domain concepts, responsibilities, relationships, data ownership, and invariants without committing to code-level structure. Use a class, entity, or conceptual diagram where useful. For every class and interface in a class diagram, list its primary responsibility in a titled and labeled register, as required by DOC-CLASS-001. Reference that register from the logical view and keep it synchronized with the diagram. Under DOC-LINK-001, give each class or interface row a stable label and link class names in diagrams and prose to that row.

9.3 Project Status

No project classes, interfaces, or allocations are defined. See Q-03, owned by Nkeh P. Boh.

Chapter 10

Process View

10.1 Chapter Responsibility

Explain how the system behaves while it is running.
Expected result: A runtime model that explains coordination and observable behavior.

10.2 How to Use This Chapter

Describe runtime behavior, concurrency, state transitions, workflows, error handling, and significant sequences. Use activity, sequence, or state diagrams when prose is insufficient.

10.3 Project Status

Execution contexts, scheduling, states, and concurrency are unresolved. See Q-05, owned by Nkeh P. Boh.

Chapter 11

Development View

11.1 Chapter Responsibility

Map the design to source organization and build artifacts.
Expected result: A code organization that developers can implement consistently.

11.2 How to Use This Chapter

Show how source code, modules, services, libraries, and build artifacts are organized. State dependency rules so that implementers do not create forbidden coupling.

11.3 Project Status

Firmware layers, dependencies, and development structure are unresolved. See Q-03, owned by Nkeh P. Boh.

Chapter 12

Physical View

12.1 Chapter Responsibility

Describe where the system runs and how deployed elements communicate.
Expected result: A deployment model grounded in the operating environment.

12.2 How to Use This Chapter

Describe deployment nodes, networks, environments, runtime dependencies, trust boundaries, data stores, and operations constraints. Include a deployment diagram for distributed systems.

12.3 Project Status

Processors, boards, physical deployment, and hardware revisions are unresolved. See Q-03, owned by Nkeh P. Boh.

Chapter 13

Implementation

13.1 Chapter Responsibility

Make the architecture reproducible through concrete implementation guidance.
Expected result: A repeatable path from design to a working implementation.

13.2 How to Use This Chapter

Document the selected technology baseline, build and run instructions, coding conventions that enforce architecture, configuration, and definition of done. Keep design rationale in Architecture Overview.

13.3 Project Status

Implementation is outside the authorized work. Technical toolchain choices remain unresolved. See Q-06, owned by Nkeh P. Boh.

13.4 Document Build

Run `make -f ftdi2Build.mk pdf` from the project directory. The build runs PDFLaTeX, BibTeX, MakeGlossaries, MakeIndex, and the final PDFLaTeX passes. The required tools must be on PATH. There is no authorized firmware build or product test command. Document-build results do not establish implementation verification or stakeholder validation.

Chapter 14

Tests

14.1 Chapter Responsibility

Define repeatable procedures that produce objective evidence.
Expected result: Executable test specifications with observable pass criteria.

14.2 How to Use This Chapter

Define repeatable test cases, levels, environments, data, entry conditions, procedures, expected results, and result recording. Tests produce evidence; they do not replace verification or validation.

14.3 Project Status

No product test procedure or execution result has been supplied. See Q-10, owned by Nkeh P. Boh.

Chapter 15

Traceability

15.1 Chapter Responsibility

Maintain the canonical links from stakeholder need to release evidence.
Expected result: Visible coverage and identifiable gaps across the lifecycle.

15.2 How to Use This Chapter

Maintain the canonical mapping from stakeholder need or approved constraint through requirement, design decision, implementation, test, verification, validation, and release evidence. Review both missing backward links and missing forward links.

15.3 Project Status

No project product-artifact relationships exist yet. Empty registers do not establish coverage. See Q-10, owned by Nkeh P. Boh.

The canonical traceability register currently contains zero project rows. When supported artifacts are introduced, maintain reciprocal story, requirement, use-case, architecture, test, verification, and validation mappings. Questions link to their owning chapters without asserting product-artifact coverage.

Chapter 16

Verification

16.1 Chapter Responsibility

Show that each requirement conforms to its specification.
Expected result: A requirement-by-requirement account of demonstrated compliance.

16.2 How to Use This Chapter

Show that each requirement conforms to its specification using test, inspection, analysis, or demonstration. Record the method, evidence reference, responsible reviewer, result, and any waiver. Verification answers “Did we build the system right?”

16.3 Project Status

No product verification has been performed or claimed. See Q-10, owned by Nkeh P. Boh.

Chapter 17

Validation

17.1 Chapter Responsibility

Show that the completed system meets stakeholder needs in its intended context.
Expected result: Evidence that stakeholders can achieve their intended outcomes.

17.2 How to Use This Chapter

Evaluate whether the completed system solves stakeholder needs in its intended context. Define realistic scenarios, participant roles, success criteria, observations, and disposition of requested changes. Validation answers "Did we build the right system?"

17.3 Project Status

No stakeholder validation has been performed or claimed. See Q-10, owned by Nkeh P. Boh.

Chapter 18

Release

18.1 Chapter Responsibility

Identify the exact deliverable and establish its readiness for authorized release.
Expected result: A controlled release package presented to the release authority.

18.2 How to Use This Chapter

Identify the release and define contents, approvals, evidence, known limitations, deployment instructions, rollback or recovery approach, and support ownership.

18.3 Initialization Acceptance and Publication

On November 12, 2026, Nkeh P. Boh accepted the initialized architecture documentation project and explicitly authorized committing its sources on `main` and pushing to boh-prog/EmbeddedSoftware_Architecture. The existing repository history is retained. This authorization resolves Q-11; it does not approve missing technical decisions, product requirements, implementation, verification, validation, or a product release. Questions Q-01 through Q-10 remain open in the owned-question register.

The publication package contains the authoritative LaTeX, bibliography, separate TikZ sources, required class and bibliography style, build scripts, AI discovery pointers, template manifest, review report, and compiled PDF. Temporary review renders and toolchain files are not publication sources. The initial publication uses main under the owner's explicit instruction. Every later change uses a branch and pull request, as recorded in change control.

Appendix A

Document Authoring Guide

A.1 Chapter Responsibility

Keep the manual consistent, maintainable, and reproducible.
Expected result: An engineering document whose sources and references remain usable.

A.2 How to Use This Appendix

Use this appendix to maintain authoring conventions, repository locations, diagram sources, review rules, tool installation, and PDF build instructions. Update it whenever the document structure or toolchain changes.

A.3 Template Provenance

This template was developed by **Darian Muresan, Ph.D.**, in conjunction with the AI agents **OpenAI Codex** and **Anthropic Claude Code**. Questions, corrections, and proposed improvements may be sent to dmuresan@stevens.edu, darian@dmmd.net, or darian@pictomic.com.

Keep the template attribution distinct from the project-specific document owner and organization. A derived project may replace the project metadata while retaining this provenance statement for the underlying template.

A.4 AI-Assisted Authoring

The authoritative instructions for AI-assisted architecture work, including skill selection, limited context loading, workflows, example prompts, completion checks, and human-review boundaries, are contained in Chapter C. This appendix retains installation, build, file, and LaTeX authoring guidance that applies to both human authors and AI agents.

A.5 File Naming

Every project-owned file created for this document shall begin with the project file namespace, followed by a Pascal-style responsibility name. The namespace is defined once in `ftdi2Manual.tex`. The defining macro is `FileNamespace`, and this template sets it to `ftdi2`. For example, `ftdi2Requirements.tex` contains the requirements chapter. The convention applies to chapter files, diagram sources, generated project figures, evidence files, and other project-owned artifacts. A tool-mandated or externally maintained file name may remain an explicitly documented exception, but shall not establish a second project naming convention.

Before creating or renaming files for a derived project, an AI agent shall ask an authorized human for the project file namespace or confirm a namespace that is already explicitly configured. The agent shall not invent one. After it is confirmed, replace `\FileNamespace` and apply the namespace consistently to every new project-owned file.

A.5.1 Document-Authoring Requirements

The following requirements govern the template itself and every project derived from it. They are verified by repository inspection and by compiling the document.

Table A.1: Document-Authoring Requirements

ID	Requirement	Verification
DOC-FILE-001	Except for an explicitly documented tool-mandated discovery file, every new project-owned file shall be named `<nameSpace><FileName>.<extension>`, using the human-approved project namespace and a descriptive Pascal-style file name.	Inspect all newly created project files.
DOC-TIKZ-001	Every TikZ source shall be stored under `TikZ/<includingFileBaseName>/` and shall be named `<nameSpace><DrawingName>.tex`. The directory name shall exactly match the base name of the LaTeX file that contains its `input` command.	Compare each TikZ path with its including LaTeX file and compile the document.
DOC-TIKZ-002	Every arrow, connector, and leader line in every TikZ drawing shall use only horizontal and vertical segments, with right-angle turns. Diagonal segments are prohibited. Connector text shall be clear of boxes, nodes, and line strokes; move the label or increase node spacing as needed. This routing rule does not alter the geometry of diagram shapes.	Inspect every drawing in the compiled PDF at presentation size for diagonal line segments and overlapping labels.

Document-Authoring Requirements (continued)

ID	Requirement	Verification
DOC-CLASS-001	Every class and interface represented in an architecture class diagram shall have one entry in a titled, labeled class-responsibility register. Each entry shall state its primary responsibility. Keep the register complete when the diagram changes.	Compare each class diagram with its class-responsibility register and follow the table reference in the compiled PDF.
DOC-LINK-001	Link references to defined architecture artifacts to their authoritative definitions wherever the document format permits. This includes requirements, stories, use cases, architecture decisions, figures, tables, and named classes or interfaces. The class-responsibility register is the definition target for a class or interface: give each register row a stable label and link its name in diagrams and prose to that row. Use a chapter or section link when an individual target is unavailable, and avoid repeating the definition at the reference site.	Follow representative references in the compiled PDF, including class names in diagrams and architecture IDs, and confirm that each reaches the intended definition or responsibility row.
DOC-AI-001	Before creating or renaming any project-owned file, an AI agent shall ask for or confirm the project `<nameSpace>` and shall apply the confirmed namespace to all files it creates.	Review the AI interaction record and inspect created file names.
DOC-AI-002	Every repository-level AI instruction file shall point to `ftdi2AIAgentGuide.tex` and the relevant authoritative LaTeX sources, shall instruct the agent to read them before architecture-related work, and shall not duplicate their rules. Tool-mandated discovery file names are documented exceptions to DOC-FILE-001.	Inspect each repository AI instruction file and compare it with the authoritative LaTeX guidance.
DOC-AI-003	An AI agent editing architecture shall link references to defined AI skills, Context, Steps, Acceptance, requirements, figures, tables, sections, and chapters to their authoritative definitions. Each AIS reference shall target its individual register row, and each row and skill definition shall provide reciprocal links. Literal prompts, code, and external quotations may use plain text with links in the surrounding prose.	Inspect changed references and follow their links in the built PDF, including both directions between AIS rows and definitions.

Document-Authoring Requirements (continued)

ID	Requirement	Verification
DOC-AI-004	AI-assisted edits shall use the established AI Skill names and `aia` role identifiers defined in Consistent terminology, including distinct suffixes for parallel instances. Human roles shall retain their human names.	Inspect changed prose, prompts, tables, and diagrams for consistent identifiers.
DOC-BIB-001	Every external source reference shall use a stable BibTeX key in `ftdi2References.bib` and a `\cite` command, including references in footnotes. Store source titles, organizations, URLs, revisions or publication dates when available, and access dates for web sources in the bibliography record.	Inspect source references, run BibTeX, and check that citations resolve to the document bibliography.

A.6 One Responsibility per Chapter

Every authored chapter and appendix shall begin with a numbered *Chapter Responsibility* section, after its chapter label and index entries. State the chapter's primary responsibility and expected result. Give the section a stable label so other chapters can link to its scope. Keep this summary aligned with the chapter content when tailoring the template.

A chapter answers one major engineering question. When content belongs to another responsibility, link to its authoritative chapter instead of copying it. In particular, tests define procedures, verification records requirement compliance, validation evaluates stakeholder use, and release decides whether the evidence is sufficient.

A.7 Adding a Chapter

Create the namespaced file, begin it with a chapter title and stable chapter label, add useful index terms and a Chapter Responsibility section, and include it once from `ftdi2Manual.tex`. Add or change artifact mappings only in the canonical traceability matrix.

```
\chapter{Example Chapter}
\ChapterLabel{ExampleChapter}
\index{example}
\index{Chapter!Example Chapter}

\section{Chapter Responsibility}
\label{Section:ExampleChapterResponsibility}
State the primary responsibility of this chapter.
```

```
\noindent\textbf{Expected result:} State its reviewable output.

\section{How to Use This Chapter}
Chapter content goes here.
```

A.8 Diagram Authoring with TikZ

TikZ is the diagram-authoring tool for this template. Authors shall use TikZ for architecture diagrams because the editable source remains in LaTeX, text uses the document fonts, output stays vector based, and diagram changes can be reviewed with the surrounding specification. The main document loads TikZ and a common set of libraries for arrows, positioning, geometric shapes, fitting, and coordinate calculations.

Every TikZ source shall use the following directory hierarchy:

`TikZ/<includingFileBaseName>/<nameSpace><DrawingName>.tex`

The directory name shall exactly match the base name of the LaTeX file that contains the diagram's `\input` command. The TikZ file name shall begin with the configured project namespace and describe the drawing. For example, the Introduction chapter includes its V-model from:

`TikZ/ftdi2Introduction/ftdi2VModel.tex`

Create additional including-file directories as required; do not place TikZ sources beside chapter files or in generated-image directories. Connector routing and label placement shall satisfy DOC-TIKZ-002.

The TikZ source file should contain the `tikzpicture` environment. The owning chapter shall include that source in a numbered figure, provide a descriptive caption, place a stable label after the caption, and reference the figure from the chapter text:

```
As shown in Figure~\ref{Figure:HighLevelUseCase}, ...

\begin{figure}[htbp]
  \centering
  \input{TikZ/ftdi2UseCases/ftdi2UseCaseDiagram.tex}
  \caption{High-level use-case diagram.}
  \label{Figure:HighLevelUseCase}
\end{figure}
```

A.9 Building the PDF

TikZ diagrams compile directly when their source files are included by the owning chapters. The toolchain requires:

- a LaTeX distribution containing TikZ/PGF, `pdflatex`, `bibtex`, and `latexmk`;
- the `makeglossaries` and `makeindex` commands.

A.9.1 Installing the Tools on macOS

The full MacTeX distribution is recommended because it includes the LaTeX tools and packages used by this template. Complete these steps from an administrator account:

1. Open Terminal and install Apple's command-line tools, which provide `make`:

   ```
   xcode-select --install
   ```

   Complete the installer dialog before continuing.

2. Install Homebrew by following the current instructions in [4]. Then install MacTeX, which includes TikZ:

   ```
   brew install --cask mactex
   ```

   MacTeX may instead be downloaded using the distribution instructions in [5].

3. Close and reopen Terminal so the MacTeX command-line tools are added to the system path.

4. Verify that every required command is available:

   ```
   command -v make pdflatex bibtex \
     makeglossaries makeindex latexmk
   kpsewhich tikz.sty
   ```

   Each command shall print a path. A missing line identifies the tool or TikZ package that still needs to be installed or added to the system path.

A.9.2 Installing the Tools on Windows

Use a current Windows 10 or Windows 11 computer and complete these steps:

1. Download and run the Basic MiKTeX Installer following [6]. Choose a private per-user installation when appropriate and select *Always install missing packages on-the-fly* for a computer with Internet access.

2. Open MiKTeX Console, check for updates, and install them. If `latexmk` is not found during verification, use the Packages page in MiKTeX Console to install the `latexmk` package.

3. Close and reopen PowerShell, then verify the required commands:

   ```
   Get-Command pdflatex,bibtex
   Get-Command makeglossaries,makeindex,latexmk
   kpsewhich tikz.sty
   ```

Each tool shall produce a command record. A command-not-found error identifies a missing installation or system-path entry.

On an organization-managed computer, authors shall ask the system administrator before installing software or changing a managed execution policy.

A.9.3 Installing the Tools on Ubuntu or Debian Linux

Open a terminal and install TikZ/PGF, GNU Make, Latexmk, and a TeX Live set that contains the other packages used by this template:

```
sudo apt update
sudo apt install make latexmk texlive-pictures \
  texlive-latex-extra \
  texlive-fonts-recommended texlive-bibtex-extra \
  texlive-extra-utils
```

Verify the required commands:

```
command -v make pdflatex bibtex \
  makeglossaries makeindex latexmk
kpsewhich tikz.sty
```

For another Linux distribution, install the equivalent TikZ/PGF, GNU Make, Latexmk, and full or extended TeX Live packages with its package manager.

A.9.4 AI-Agent Installation

An AI coding agent can inspect the host, identify missing commands, request approval for system changes, install the appropriate packages, and verify the build. Give the agent the project directory and the following reusable request:

```
In this LaTeX project, inspect the build files and determine
which operating system and package manager are available.
Verify TikZ with ‘kpsewhich tikz.sty‘, and verify make,
pdflatex, bibtex, makeglossaries, makeindex, and latexmk.
Install only missing tools using the
platform’s standard package manager. Ask for approval before
administrator-level, system-wide, or networked installation.
Do not replace project sources or remove user files. After
installation, compile included TikZ sources, run the full platform
build, inspect the log for errors, unresolved
references, and overfull boxes, and report the generated PDF.
```

If the agent cannot install software, ask it to report the exact missing command, the attempted verification, and platform-specific installation instructions. Never give an agent an unrestricted instruction to delete build directories or modify system security policy.

A.9.5 Building on macOS and Linux

From the template directory, run:

```
make -f ftdi2Build.mk pdf
```

Use `make -f ftdi2Build.mk clean` to remove generated build files while preserving authored sources and the final PDF.

A.9.6 Building with Windows PowerShell

From PowerShell in the template directory, run:

```
.\ftdi2Build.ps1 -Action pdf
```

Use `-Action clean` in place of `-Action pdf` to remove generated build files. If local execution policy prevents scripts from running, use an approved organizational policy or allow scripts only for the current PowerShell process:

```
Set-ExecutionPolicy -Scope Process Bypass
.\ftdi2Build.ps1 -Action pdf
```

A.9.7 Manual Build Sequence

When the platform build script is unavailable, run the following commands from the template directory. The final two LaTeX passes resolve tables of contents, citations, glossary links, the index, and cross-references.

Included TikZ sources require no separate rendering command; `pdflatex` compiles them with their owning chapters:

```
pdflatex -interaction=nonstopmode -halt-on-error ftdi2Manual.tex
bibtex ftdi2Manual
makeglossaries ftdi2Manual
makeindex ftdi2Manual.idx
pdflatex -interaction=nonstopmode -halt-on-error ftdi2Manual.tex
pdflatex -interaction=nonstopmode -halt-on-error ftdi2Manual.tex
```

A.9.8 Build Verification and Troubleshooting

Confirm that the output PDF opens, all expected chapters and appendices appear, the glossary and index are populated, hyperlinks reach their targets, and no table or figure is clipped. Search `ftdi2Manual.log` for fatal errors, undefined references, missing files, and overfull boxes. Common remedies are:

- open a new terminal after installation so its search path is refreshed;

- on macOS, verify that `/Library/TeX/texbin` is on the search path;
- rerun the complete build after adding citations, glossary entries, index entries, figures, labels, or references;
- compile and inspect the owning chapter after changing a TikZ source;
- install the named LaTeX package when the log reports a missing `.sty` file.

A.10 Authoring Rules

- Add the newest approved change to the top of the document history.
- Introduce specialized terms through the glossary and use them consistently.
- Give every controlled artifact a stable explicit ID.
- Use `shall` only for binding requirements.
- Keep normative requirements separate from explanation and examples.
- Apply the approved project namespace to every new project-owned file as specified in the document-authoring requirements.
- Use TikZ for all diagrams. Store every source in `TikZ/<includingFileBaseName>/` and name it `<nameSpace><DrawingName>.tex`.
- Include each TikZ source from its owning chapter in a numbered figure with a caption and stable label, and reference that figure in the chapter text.
- Compile and inspect the PDF after structural or table changes.

A.11 Citations and Bibliography

Store all external source reference records in `ftdi2References.bib`, including standards, publications, web documentation, and installation guides, as required by DOC-BIB-001. Cite each source with a stable BibTeX key; references in footnotes use the same citation command. Keep source titles, organizations, URLs, revisions or publication dates when available, and web access dates in the bibliography record. Do not embed source URLs directly in prose or footnotes. Internal cross-references and contact email links retain their existing link commands.

Cite a source from a chapter:

```
\cite{ISO29148}
```

The main file applies the style and prints one document-level bibliography after the appendices and glossary, immediately before the index:

```
\bibliographystyle{IEEEtran}
\bibliography{ftdi2References}
```

Both platform build scripts run BibTeX automatically.

Bibliography entries support claims and definitions; they do not replace project-owned requirements, architecture decisions, test procedures, or approval evidence.

A.12 Selective Builds

For a large project, LaTeX's `\includeonly` command may select chapters during drafting. The main file contains a commented example:

```
% \includeonly{
%   ftdi2Prologue,
%   ftdi2Requirements,
%   ftdi2Tests
% }
```

Remove the leading comment characters and list include names without the `.tex` extension. LaTeX can retain cross-references and citations from omitted chapters by reading their existing auxiliary files, but those files may be absent or stale. Selective builds are therefore drafting aids only. Hyperref may also report missing destinations for links into deliberately omitted chapters; these draft warnings shall disappear in the full build. Comment out the complete `\includeonly` block and run a full build before verification or release so missing chapters, references, glossary entries, citations, and evidence cannot be hidden.

A.13 Project Naming Exceptions and Inherited Examples

`AGENTS.md` and `SKILL.md` are tool discovery exceptions. `.gitignore` is a Git tool filename. The pre-existing `README.md` is retained. `cornell.cls` and `IEEEtran.bst` retain their externally maintained names and contents. Git metadata and pre-existing operating-system metadata are not authored architecture sources. All newly authored project sources, build scripts, reports, and review images use the confirmed namespace.

The guide appendices retain generic instructional examples, bracketed prompt fields, and profile checklist patterns. Those are guidance, not project facts. The project core chapters contain no adopted sample service-request artifacts.

Appendix B

Embedded Systems Tailoring Profile

B.1 Chapter Responsibility

Add the concerns needed when software interacts with constrained hardware.
Expected result: The core architecture tailored to the embedded operating context.

B.2 How to Use This Appendix

This appendix tailors the generic architecture template for systems in which software interacts directly with dedicated hardware, constrained computing resources, physical processes, or real-time events. It applies to bare-metal firmware, real-time operating system (RTOS) products, embedded Linux systems, programmable logic, hardware–software products, and other cyber-physical systems.

The profile augments the core template; it does not create a second architecture specification. Project-specific requirements, interfaces, decisions, diagrams, tests, and evidence shall remain in their authoritative core chapters. When this profile applies, retain this appendix as the authoritative tailoring guide, record the applicability decision in Chapter 2, and use the additional concerns below when creating and reviewing the core content. Remove this appendix from an instantiated project only when an authorized human records that the profile is not applicable.

B.3 Applicability

Apply this profile when one or more of the following conditions is present:

- software executes on a dedicated processor, microcontroller, system on chip, field-programmable device, or purpose-built computing module;
- behavior depends on interrupts, deadlines, deterministic scheduling, physical signals, sensors, actuators, or control loops;

- CPU, memory, storage, bus bandwidth, energy, or thermal resources are materially constrained;
- the product requires hardware bring-up, boot firmware, board support, device drivers, manufacturing provisioning, calibration, or field firmware updates; or
- safety, reliability, environmental, physical-security, certification, or component-lifecycle concerns affect the architecture.

The applicability decision shall identify the embedded platform class, the reason the profile applies, the hardware revisions in scope, and any other domain profiles used concurrently. A project may combine this profile with a future safety-critical, high-security, cloud, mobile, or machine-learning profile. Each profile remains guidance; controlled project facts stay in the core chapters.

B.4 Core-Chapter Tailoring Map

Use Table B.1 to place each embedded concern in one authoritative location.

Table B.1: Embedded-Systems Core-Chapter Tailoring Map

Core Chapter	**Embedded-System Additions**
Development Plan	Record the profile decision, hardware and firmware roles, toolchain ownership, hardware availability, prototypes, certification gates, manufacturing dependencies, and component-lifecycle risks.
System Context	Define the hardware–software boundary, physical environment, sensors, actuators, operators, service tools, external buses, power sources, and upstream or downstream systems.
User Stories and Use Cases	Include installation, startup, shutdown, calibration, diagnostics, maintenance, update, recovery, degraded operation, manufacturing, and service interactions where applicable.
Requirements	Add timing, resource, power, interface, boot, recovery, safety, security, reliability, environmental, diagnostics, update, manufacturing, and regulatory obligations.
Architecture Views	Document hardware–software allocation, firmware layers, execution contexts, states, timing, memory, deployment, data and signal flow, power modes, faults, and recovery.
Implementation	Identify processor and board revisions, boot chain, BSP, HAL, RTOS or operating system, compiler, linker, SDK, libraries, generated code, linker scripts, build variants, image formats, and programming tools.
Tests, Verification, and Validation	Add target execution, static analysis, timing analysis, SIL, PIL, HIL, fault injection, power, environmental, hardware-revision, update, recovery, and retained-evidence activities.

Embedded-Systems Core-Chapter Tailoring Map (continued)

Core Chapter	Embedded-System Additions
Release	Identify released hardware revisions, firmware images, hashes, configuration data, programming instructions, compatibility, rollback, manufacturing release, service provisions, and known limitations.

B.5 Hardware–Software Boundary and Allocation

The System Context and architecture chapters shall identify every processor, memory device, programmable component, peripheral, sensor, actuator, bus, power domain, and external physical interface that materially constrains the software. Define which behavior is implemented in hardware, firmware, programmable logic, operating-system services, application software, or an external system. Record alternatives and trade-offs for significant allocation decisions.

For each allocated function, identify its driving requirements, execution platform, inputs, outputs, timing needs, failure behavior, update boundary, and verification method. Do not treat a vendor module, BSP, HAL, RTOS, device driver, or generated component as an unexamined implementation detail when it affects required behavior.

B.6 Embedded Requirement Categories

Table B.2 is a checklist of common embedded requirement categories. It does not introduce new requirement ID families. Classify each requirement in the existing family that best describes its primary obligation. Propose a new family only through the controlled procedure in Chapter 5. Apply the domain-profile translation rule in Section 5.7: every applicable embedded concern shall map to a project requirement, an existing requirement, or an approved Not Applicable decision with rationale.

Table B.2: Embedded-System Requirement Categories

Category	Questions the Requirements Shall Resolve
Functional behavior	What observable control, sensing, computation, communication, diagnostic, maintenance, and recovery behavior is required?
Timing and real time	What deadlines, periods, latency, jitter, sequencing, startup, shutdown, and worst-case response limits apply?
Processing resources	What CPU utilization, execution-time, core-affinity, interrupt-latency, and scheduling limits apply?
Memory and storage	What flash, RAM, stack, heap, nonvolatile storage, retention, endurance, and memory-protection limits apply?
Power and thermal	What operating, sleep, peak, average, battery-life, wake-up, brownout, thermal, and power-transition behavior is required?

Embedded-System Requirement Categories (continued)

Category	Questions the Requirements Shall Resolve
Hardware interfaces	What electrical levels, pin assignments, connectors, buses, protocols, rates, units, ranges, scaling, resolution, endianness, and error behavior apply?
Boot and initialization	What boot sources, ordering, self-tests, startup times, configuration checks, and failure responses are required?
Reset and recovery	What events cause reset, retry, failover, degraded mode, safe state, rollback, or operator intervention?
Safety	What hazards, safety functions, safe states, diagnostic coverage, independence, and integrity constraints apply?
Security	What root of trust, secure boot, image authentication, device identity, key provisioning, access control, debug-port, anti-rollback, and physical-attack provisions apply?
Reliability and availability	What service life, duty cycle, fault tolerance, watchdog, data-integrity, availability, maintainability, and recovery targets apply?
Environmental	What temperature, humidity, vibration, shock, altitude, electromagnetic, contamination, ingress, and storage conditions apply?
Diagnostics and observability	What logs, counters, health indicators, fault codes, traces, test points, telemetry, and service interfaces are required?
Calibration and configuration	What values are configurable or calibrated; who controls them; how are range, integrity, compatibility, and provenance maintained?
Firmware update	What transport, authentication, atomicity, compatibility, interruption, rollback, recovery, audit, and field-service behavior applies?
Manufacturing and provisioning	What programming, serialization, key injection, calibration, end-of-line testing, traceability, and acceptance data is required?
Regulatory and certification	What standards, evidence, configuration controls, independence, analysis, and approval obligations apply?
Component lifecycle	What approved components, alternates, lead-time, obsolescence, silicon errata, vendor support, and redesign constraints apply?

Each embedded requirement shall retain the normal priority, lifecycle status, In Version value, use-case mapping, and test-case mapping. Quantitative limits shall state units, applicable operating conditions, tolerance, and verification method. Avoid terms such as “fast,” “low power,” or “robust” without a measurable acceptance boundary.

B.7 Required Architecture Views

Select views according to risk and complexity, and explain any omitted view that would normally apply. Create diagrams with TikZ and store them under the owning core chapter.

Table B.3: Candidate Embedded-System Architecture Views

View	Content
Hardware block view	Processing, memory, programmable logic, peripherals, clock, reset, power, sensors, actuators, and external connections.
Hardware–software allocation	Required functions allocated to hardware, firmware layers, programmable logic, or external systems.
Firmware layering	Bootloader, BSP, HAL, drivers, RTOS or kernel, middleware, services, application, and diagnostic boundaries.
Boot and update sequence	Trust chain, initialization order, self-tests, configuration loading, update, rollback, and recovery paths.
Execution-context view	Cores, processes, tasks, threads, interrupt service routines, priorities, scheduling, synchronization, and shared resources.
State and mode view	Operational, maintenance, calibration, degraded, safe, sleep, update, and fault states with permitted transitions.
Timing view	Periods, deadlines, latency, jitter, sequencing, bus transactions, control-loop timing, and worst-case paths.
Memory view	Address spaces, partitions, executable regions, stacks, heaps, DMA buffers, retained data, memory protection, and nonvolatile layout.
Data and signal flow	Units, scaling, sample rates, filtering, validity, ownership, persistence, and error propagation.
Deployment view	Firmware images and configuration deployed to processors, cores, boards, hardware revisions, and manufacturing or field tools.
Power and thermal view	Power domains, modes, transitions, wake sources, consumers, thermal limits, throttling, and shutdown behavior.
Fault-containment view	Fault-detection boundaries, watchdogs, redundancy, degraded operation, safe states, containment, and recovery.

B.8 Interfaces and Protocols

For every hardware or communication interface, document the physical and logical contract together. Include connector and pin identity, direction, electrical characteristics, protocol and version, topology, addressing, message or register definition, data type, units, range, resolution, scaling, endianness, update rate, timing, initialization, ownership, concurrency, timeouts, integrity checks, invalid values, error handling, compatibility, and verification method where applicable.

An interface table or referenced interface-control document shall have a controlled identifier and version. Each interface requirement and test shall link to that authority. Do not infer an electrical limit, register behavior, reserved bit, or protocol tolerance without controlled evidence.

B.9 Resource Budgets

Maintain resource budgets whenever architecture feasibility depends on a finite embedded resource. Table B.4 is a generic pattern; create one row for each controlled allocation and reconcile component allocations with the system total.

Table B.4: Generic Embedded Resource Budget

Resource	**Owner**	**Allocation**	**Measured or Estimated**	**Requirement / Evidence**
CPU time	[task or component]	[limit and units]	[value, method, conditions]	[requirement and test IDs]
Flash	[image or component]	[bytes or percent]	[map-file value]	[requirement and evidence]
RAM	[task or component]	[bytes or percent]	[static and peak value]	[requirement and evidence]
Stack	[task or context]	[bytes]	[worst observed or analyzed]	[requirement and evidence]
Bus bandwidth	[interface or flow]	[rate and utilization]	[value and conditions]	[requirement and test IDs]
Power or energy	[mode or component]	[power, current, or energy]	[value and conditions]	[requirement and test IDs]
Thermal margin	[component or mode]	[temperature or margin]	[value and conditions]	[requirement and evidence]

Every budget shall state the measurement or estimation conditions, margin policy, uncertainty, owner, update trigger, and response to an exceeded allocation. Budget evidence shall distinguish analysis, simulation, measurement, and worst-case guarantee.

B.10 Safety, Reliability, and Fault Management

Identify hazards, failure modes, fault sources, detection mechanisms, containment boundaries, watchdogs, redundancy, degraded modes, safe states, recovery behavior, diagnostic coverage, and residual risk where applicable. Trace every credited mechanism to requirements and verification evidence.

Do not claim a safety integrity level, diagnostic coverage, failure rate, independence, or risk acceptance without the applicable analysis and authorized approval. When a dedicated safety profile or regulated process applies, use it together with this embedded profile and record precedence for any conflicting rule.

B.11 Embedded Security

Define the trust boundary and lifecycle from manufacturing through decommissioning. Address device identity, root of trust, secure boot, image authentication, signing and key custody, provisioning, credential storage, debug and test access, privileged interfaces, update authorization, anti-rollback, protected configuration, security logging, recovery, physical attack assumptions, vulnerability response, and secure decommissioning as applicable.

Security controls shall be mapped to threats, requirements, tests, retained evidence, and operational responsibilities. Secrets and production keys shall not be placed in this architecture document, source repository, AI prompt, or test artifact.

B.12 Toolchain and Configuration Control

Record the controlled hardware revisions, processor variants, compiler and linker versions, optimization and warning settings, linker scripts, BSP, HAL, RTOS or operating-system version, SDK, libraries, code generators, programming tools, static-analysis tools, build variants, configuration sources, and image formats. Identify which outputs are generated, how they are reproduced, and which tool qualifications or validations apply.

A released firmware image shall be traceable to source revision, configuration, toolchain, dependencies, hardware compatibility, build procedure, image hash, and approval evidence. Changes to silicon revision, board revision, compiler, optimization, RTOS, BSP, HAL, linker script, or generated-code input shall trigger an impact assessment.

B.13 Embedded Verification and Validation

Select test environments according to the requirement and the confidence needed. The verification plan may include review, static analysis, host unit tests, software-in-the-loop (SIL), processor-in-the-loop (PIL), target tests, hardware-in-the-loop (HIL), fault injection, timing and worst-case execution analysis, stress, endurance, power, thermal, environmental, electromagnetic, hardware-revision compatibility, boot, update, rollback, and recovery testing.

Each result shall identify the hardware revision, firmware version, build configuration, tool and fixture versions, calibration state, environmental conditions, input data, procedure, expected result, actual result, retained evidence, and reviewer. Simulation or host-test evidence shall not be represented as target evidence. Validation shall use representative intended operation and stakeholders or authorized representatives.

B.14 AI Tailoring Instructions

When this profile is Applicable or TBD, an AI agent shall read this appendix with the task-specific core chapters. Before editing, it shall ask for or confirm:

- the platform class: bare metal, RTOS, embedded Linux, programmable logic, or mixed;

- processor, board, programmable-device, and hardware revisions in scope;
- boot chain, operating system or RTOS, BSP, HAL, SDK, compiler, linker, and relevant third-party components;
- external interfaces, sensors, actuators, physical environment, timing, resource, power, safety, security, regulatory, manufacturing, update, and service constraints; and
- available schematics, datasheets, interface-control documents, hazard analysis, threat analysis, test evidence, and authorized approvals.

The agent shall add embedded concerns to the initialization gap list and place project facts only in their authoritative core chapters. It shall not invent pinouts, register behavior, electrical limits, memory sizes, execution times, power measurements, hardware compatibility, safety classifications, security claims, certification evidence, or test results. It shall report missing controlled hardware evidence and every decision requiring authorized human approval.

Use the following request for an embedded project:

```
Read ftdi2AIAgentGuide.tex and ftdi2EmbeddedSystemsProfile.tex. Apply the
Embedded Systems Tailoring Profile to [Project Name]. Ask me to
confirm the project namespace, profile applicability, platform class,
hardware revisions, toolchain, interfaces, constraints, applicable
standards, and available evidence. Add embedded concerns to the gap
list before editing. After human review, place project-specific
requirements, architecture, diagrams, tests, and evidence only in
their authoritative core chapters. Do not invent hardware facts or
verification results. Build the complete document, inspect the log,
and visually inspect the affected PDF pages.
```

B.15 Embedded-Profile Completion Checklist

Before completing embedded tailoring, confirm that:

- profile applicability and rationale are recorded;
- the hardware–software boundary and function allocation are explicit;
- timing, resources, power, interfaces, boot, recovery, safety, security, environmental, update, manufacturing, and lifecycle concerns were assessed;
- required embedded views and resource budgets exist or have documented omission rationales;
- hardware revisions, firmware versions, toolchain, build variants, and released images are configuration controlled;

- requirements, use cases, tests, traceability, verification, validation, and release records have reciprocal links; and
- the build succeeds and affected tables, diagrams, links, and page breaks have been visually inspected.

Appendix C

AI Agent Usage Guide

C.1 Chapter Responsibility

Define the authoritative inputs and acceptance criteria for AI-assisted tasks.
Expected result: A bounded task specification that an AI agent can follow and review.

C.2 How to Use This Appendix

This appendix is the entry point for an AI agent that creates, reviews, or maintains this architecture specification. The agent shall read this file first, identify the requested task in the AI Skills Register, and load only the authoritative sources and linked context listed for that task. It shall expand the context when a referenced dependency, cross-cutting decision, or inconsistency requires it.

This appendix defines what each task requires: authoritative inputs, context, task-specific steps, and acceptance criteria. Use Chapter D to execute those tasks, whether the output is an architecture document, analysis, code, tests, or evidence. That appendix owns planning, the agent-count decision, delegation, integration, verification, review, and validation handoffs. Its single-agent and delegated paths inherit this appendix's skill definitions and governing rules. Figure D.2 shows the handoff to Chapter D and the checks against this appendix's Acceptance and completion checklist.

This appendix replaces substantive AI-oriented Markdown instructions. A thin repository bootstrap may point an AI tool to this appendix, but its routing tables, context packs, task steps, prompts, and checklists are maintained only as LaTeX so they remain visible in the architecture book and versioned with the specification.

C.3 Governing Principles

The authored LaTeX files and editable diagram sources are the source of truth. TikZ is the required source format for diagrams. The generated PDF is a review artifact and shall not be edited directly. The agent shall preserve one authoritative location for each fact and use links instead of duplicating content across chapters.

An AI agent is a drafting, analysis, consistency-checking, and build assistant. It shall use supplied project evidence, identify missing information, expose material assumptions, and distinguish proposals from approved decisions. It shall not invent requirements approval, implementation status, test results, verification evidence, validation evidence, waivers, or release authorization.

Consistent terminology. Use one established term for each concept throughout requirements, prose, headings, tables, diagrams, prompts, code identifiers where applicable, and reports to the human owner. Reuse the user's chosen terms and the authoritative glossary or existing definitions. Do not introduce synonyms merely for variety, brevity, or stylistic preference. A different term shall represent a distinct concept; define that distinction before using it. When terminology conflicts or is ambiguous, identify the conflict and obtain clarification rather than silently choosing or inventing another term. Before reporting completion, check changed content for alternate names for the same concept and normalize them to the established term. Preserve external quotations and tool-mandated names; explain their mapping to project terms when needed.
Use the prefix `aia` (AI agent) for AI agent role identifiers: `aiaCoordinator`, `aiaDeveloper`, and `aiaReviewer`. Parallel instances use `aiaDeveloperA`, `aiaDeveloperB`, and `aiaDeveloperC`. Human roles retain their human names without this prefix. These identifiers describe responsibilities; one agent may perform several roles. Use these names throughout prose, prompts, figures, and reports under DOC-AI-004; responsibilities are defined in Responsibilities and Handoffs.

Use the skill labels consistently: Context identifies what the agent needs to read; Steps identify what it must do; Acceptance identifies the criteria for completing the task. Use these same labels in the AI Skills Register and skill definitions. Do not substitute Guidance for Context or Procedure for Steps. Steps and Acceptance are distinct concepts and shall not be presented as synonyms.

Linked architecture references. An AI agent editing the architecture shall satisfy DOC-AI-003: link every reference to an existing AI skill, Context, Steps, Acceptance, requirement, figure, table, section, or chapter to its authoritative definition. A skill reference shall link to that AIS's individual row in Table C.2, rather than only to the table as a whole. Each row shall link to the skill definition and its Context, Steps, and Acceptance; the skill definition shall link back to its row. Preserve readable reference text and stable labels. Literal prompts, code, and external quotations may retain plain text, with linked references in the surrounding prose. For all defined architecture artifacts, also apply DOC-LINK-001; class and interface references target their individual class-responsibility rows. Before reporting completion, check both navigation directions in the built PDF and resolve missing or incorrect targets.

C.4 AI Quick Start

1. Read this appendix and identify the requested outcome.

2. Before creating or renaming a project-owned file, ask for or confirm the project file namespace. Do not invent it; use the confirmed namespace for every file created during the task.

3. Inspect the domain-profile applicability table and the project profile decisions. Ask an authorized human to resolve any unreviewed profile selection. Load every Applicable or TBD profile affected by the task.

4. Select the closest entry in the AI Skills Register.

5. Read the primary files and context explicitly linked for each selected AI Skill (AIS). A request may require more than one AIS. Read `ftdi2Manual.tex` when document metadata, macros, inclusion order, or build structure is affected.

6. For substantial or delegated work, apply AI skill `AIS-01` and its planning and estimation context before starting the change.

7. Inspect available project evidence and report conflicts, missing facts, and consequential assumptions. Ask for a human decision when an answer would materially change scope, architecture, safety, security, cost, schedule, or release readiness.

8. Make narrowly scoped changes in the authoritative files. Preserve stable IDs and update every affected reciprocal mapping.

9. Add a linked document-history entry for an approved document change and record material AI interactions in Chapter E.

10. Execute the selected task steps using Chapter D, including its decision about one or multiple agents. For document changes, run the complete build, inspect the log, render affected PDF pages, and complete the completion checklist.

C.5 Repository AI Bootstrap

The LaTeX architecture is authoritative, but an AI tool may require a repository-level discovery file before it knows where to find these instructions. Create a thin bootstrap file for each AI tool that the project actually uses. The bootstrap shall point to this chapter and the relevant LaTeX sources; it shall not reproduce their requirements, workflows, or architectural decisions. When policy changes, update the authoritative LaTeX source rather than copying the change into each bootstrap file.

Tool-mandated discovery names are documented exceptions to the project file namespace requirement. This exception applies only to the exact filename or location required for discovery. It does not exempt files created while performing the instructed work from the namespace requirements in the document-authoring requirements.

Create only the applicable files:

Table C.1: Repository AI Bootstrap Files

File	When to Create It
`AGENTS.md`	The repository is maintained with Codex or another agent that discovers this filename.
`CLAUDE.md`	The repository is maintained with Claude Code.
`.github/copilot-instructions.md`	The repository uses repository-wide GitHub Copilot instructions.
`SKILL.md`	Reusable AI Skills are defined in this LaTeX manual; see the AI Skills Register in Table C.2. The template includes one generic skill file that directs the agent to this register.

AI Skills and tasks. An AI Skill (AIS) is a reusable definition of Context, Steps, and Acceptance. A task is a specific assignment that uses one or more AI Skills in a project context. Table C.2 is the AI Skills Register; no separate register of task assignments is required by this template. The template's generic `SKILL.md` provides a skill name and a description of when to use this manual, then directs the agent to the AI Skills Register. The agent selects the applicable AIS and reads its linked Context, Steps, and Acceptance, then executes through Chapter D. Keep the definitions in LaTeX authoritative; the skill file shall not duplicate them.

The template stores `SKILL.md` beside `AGENTS.md` and `CLAUDE.md`. Both bootstrap files explicitly direct the agent to read `SKILL.md`, which then points to this guide and the AI Skills Register. This repository flow does not require separate skill installation. Storing `SKILL.md` at the template root alone does not register it for automatic skill discovery; the explicit bootstrap pointer provides the reading route.

Each bootstrap file shall identify the root architecture source, point to this chapter, require task-appropriate context loading through the AI Skills Register, direct architecture policy changes back to LaTeX, and point to Chapter A for building and verification. It shall also state the confirmed project file namespace or direct the agent to obtain it before creating project-owned files. Use the following content as a minimal tool-neutral pattern, adapting only the discovery-specific wrapper:

```
# AI Instructions

The authoritative software architecture is maintained in
ftdi2Manual.tex. Before architecture-related work, read SKILL.md.
Follow its pointer to ftdi2AIAgentGuide.tex and use the
AI Skills Register to identify
the
additional LaTeX chapters and linked task steps required for
the task. Execute using ftdi2AIImplementationWorkflow.tex.

Treat the authored LaTeX and editable diagram sources as the
source
```

```
of truth. Do not duplicate their requirements, conventions,
workflows, or architectural decisions in this file. When policy
changes, update the authoritative LaTeX source.

Before creating project-owned files, obtain or confirm the
project
namespace required by ftdi2DocumentGuide.tex. Follow that guide's
build and verification instructions before completing the task.
```

C.6 Guide Activity

Figure C.1 starts with the AI tool's repository bootstrap: `CLAUDE.md` for Claude Code or `AGENTS.md` for Codex. Both branches then direct the agent to read the same generic `SKILL.md`. That file points to this guide and the AI Skills Register. The tool choice precedes this shared route; `SKILL.md` is not a third choice. Both branches use the same AIS definitions and Appendix D workflow. The selected AIS rows provide explicit source, Context, Steps, and Acceptance links; a context pack is the linked reading context within this appendix, not an automatically loaded tool feature. Apply the shared rules above Table C.2 and expand the reading set when an affected dependency requires it.

C.7 AI Skills Register

The file lists below are minimum starting contexts, not permission to ignore a dependency discovered during the task. A task that changes an artifact shall also inspect every register that contains a reciprocal mapping to that artifact. The `AIS-` prefix means AI Skill. These identifiers name reusable AI skills in this table, not individual assignments. Do not renumber or reuse them when the table changes.

Each row links Context, Steps, and Acceptance separately. The Steps and Acceptance links use the skill subsection names and point to the corresponding headings within those subsections. Read all three with the listed sources. Execute selected tasks using Chapter D. For every task, apply the mandatory consistency rules. For document edits or builds, also read Document-Authoring Context. For substantial or delegated work, first apply `AIS-01`. When a selected profile's concerns intersect the work, also read Domain-Profile Context. If the request changes several artifact types, select each corresponding AIS and read the union of its sources and linked context. Report an unmatched task and its proposed reading set before proceeding with consequential work.

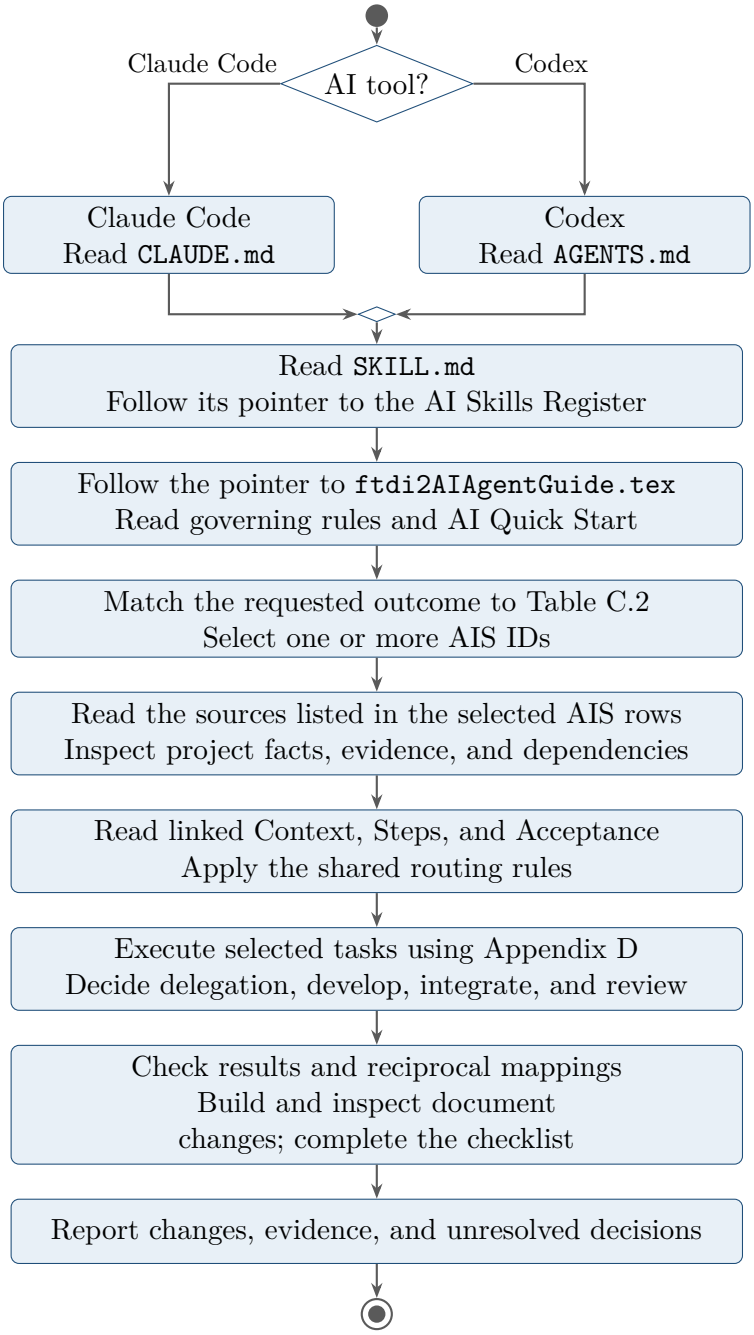


Figure C.1: AI agent activity from repository instructions through the generic skill, AIS selection, Context, Steps, Acceptance, and execution.

Table C.2: AI Skills Register

ID	AI Skill	Authoritative Sources, Context, Steps, and Acceptance
AIS-01	Estimate AI work	The request, applicable project instructions, this guide, Chapter D, the affected artifact list, dependencies, and available evidence. **Context:** Planning and Estimation. **Steps:** Estimate AI Work. **Acceptance:** Estimate AI Work.
AIS-02	Initialize a project	`ftdi2Manual.tex`, `ftdi2Prologue.tex`, `ftdi2Introduction.tex`, `ftdi2DevelopmentPlan.tex`, `ftdi2SystemContext.tex`, each proposed or selected domain profile, and the supplied project evidence. **Context:** Domain-Profile; Project initialization. **Steps:** Initialize a Project. **Acceptance:** Initialize a Project.
AIS-03	Tailor an embedded system	`ftdi2EmbeddedSystemsProfile.tex`, `ftdi2DevelopmentPlan.tex`, `ftdi2SystemContext.tex`, `ftdi2Requirements.tex`, the affected architecture and interface chapters, `ftdi2Tests.tex`, `ftdi2Traceability.tex`, and the controlled hardware and project evidence. **Context:** Domain-Profile. Select additional artifact AI Skills for the chapters being changed. **Steps:** Tailor an Embedded System. **Acceptance:** Tailor an Embedded System.
AIS-04	Create repository AI bootstrap files	`ftdi2AIAgentGuide.tex`, `ftdi2DocumentGuide.tex`, `ftdi2Manual.tex`, evidence identifying the AI tools used by the project, and each tool's required discovery filename. **Context:** Repository AI Bootstrap. **Steps:** Create Repository AI Bootstrap Files. **Acceptance:** Create Repository AI Bootstrap Files.
AIS-05	Develop user stories	`ftdi2SystemContext.tex`, `ftdi2UserStories.tex`, `ftdi2Requirements.tex`, and stakeholder evidence. **Context:** User-Story. **Steps:** Develop User Stories. **Acceptance:** Develop User Stories.
AIS-06	Add or change requirements	`ftdi2UserStories.tex`, `ftdi2Requirements.tex`, every Applicable or TBD domain profile, `ftdi2UseCases.tex`, `ftdi2Tests.tex`, `ftdi2Traceability.tex`, `ftdi2Verification.tex`, and `ftdi2Glossary.tex`. **Context:** Requirements. **Steps:** Add or Change a Requirement. **Acceptance:** Add or Change a Requirement.

Continued on next page

AI Skills Register (continued)

ID	AI Skill	Authoritative Sources, Context, Steps, and Acceptance
AIS-07	Add or change use cases	`ftdi2UserStories.tex`, `ftdi2Requirements.tex`, `ftdi2UseCases.tex`, `ftdi2Tests.tex`, and `ftdi2Traceability.tex`. **Context:** Use-Case. **Steps:** Add or Change a Use Case. **Acceptance:** Add or Change a Use Case.
AIS-08	Develop or change architecture	`ftdi2SystemContext.tex`, `ftdi2Requirements.tex`, `ftdi2ArchitectureOverview.tex`, the affected interface, logical, process, development, physical, or implementation chapters, and their `TikZ/` diagram sources. **Context:** Architecture. **Steps:** Develop or Change Architecture. **Acceptance:** Develop or Change Architecture.
AIS-09	Add or change tests	`ftdi2Requirements.tex`, `ftdi2UseCases.tex`, `ftdi2Tests.tex`, `ftdi2Traceability.tex`, `ftdi2Verification.tex`, and `ftdi2TestEvidence.tex`. **Context:** Testing and Evidence. **Steps:** Add Tests or Evidence. **Acceptance:** Add Tests or Evidence.
AIS-10	Perform verification	`ftdi2Requirements.tex`, `ftdi2Tests.tex`, `ftdi2Traceability.tex`, `ftdi2Verification.tex`, and controlled test evidence. **Context:** Testing and Evidence. **Steps:** Perform Verification. **Acceptance:** Perform Verification.
AIS-11	Perform validation	`ftdi2UserStories.tex`, `ftdi2UseCases.tex`, `ftdi2Validation.tex`, and controlled stakeholder evidence. **Context:** Testing and Evidence. **Steps:** Perform Validation. **Acceptance:** Perform Validation.
AIS-12	Prepare a release	`ftdi2Requirements.tex`, `ftdi2Traceability.tex`, `ftdi2Verification.tex`, `ftdi2Validation.tex`, `ftdi2Release.tex`, and controlled release evidence. **Context:** Release. **Steps:** Review and Prepare a Release. **Acceptance:** Review and Prepare a Release.
AIS-13	Review consistency	All registers containing the affected artifact types, `ftdi2Glossary.tex`, `ftdi2DocumentHistory.tex`, and `ftdi2Manual.tex`. **Context:** Mandatory Consistency Rules. **Steps:** Review Consistency. **Acceptance:** Review Consistency.

Continued on next page

AI Skills Register (continued)

ID	AI Skill	Authoritative Sources, Context, Steps, and Acceptance
AIS-14	Compile or troubleshoot	`ftdi2Manual.tex`, `ftdi2DocumentGuide.tex`, the affected chapter, its diagram sources, and the build log. **Context:** Document-Authoring. **Steps:** Compile or Troubleshoot. **Acceptance:** Compile or Troubleshoot.

C.8 Task-Specific Context Packs

A context pack is a reading boundary within this appendix, not a separate file. Read the packs explicitly linked in the selected AI Skills Register entries. The shared applicability rules above the table identify additional packs required by the scope of the work; do not select a pack solely by guessing from its title.

C.8.1 Planning and Estimation Context

For substantial or delegated work, identify the intended outcome, affected artifacts, acceptance criteria, dependencies, missing facts, and likely integration and review effort before starting implementation. Give a rough effort and token-use range for each proposed bounded task, with the assumptions and uncertainty behind it. Compare sequential and parallel execution, including coordination overhead. Do not present token counts as precise or guaranteed. Follow project instructions for obtaining human authorization before unusually costly work.

C.8.2 Domain-Profile Context

Determine the Applicable, Not Applicable, or TBD status of every available domain profile during initialization. An authorized human shall approve the selection and rationale. For later tasks, load every Applicable or TBD profile whose concerns intersect the requested change. A profile adds review concerns and artifact expectations; project-specific facts remain in their authoritative core chapters. When the Embedded Systems profile applies, follow Chapter B and add hardware, firmware, timing, resource, power, physical-interface, safety, security, manufacturing, update, and field-service concerns to the task context. For requirements work, apply the controlled translation rule in Section 5.7: each applicable profile concern shall map to a requirement, an existing requirement, or an approved Not Applicable decision with rationale.

C.8.3 User-Story Context

Use stakeholder evidence to identify the beneficiary, desired outcome, and acceptance criteria. Follow Chapter 4 for stable IDs, priority, and requirement links. Keep precise binding obligations in Requirements and maintain reciprocal story links in traceability.

C.8.4 Requirements Context

Establish the system boundary and stakeholder need before proposing a requirement. Keep each requirement singular, feasible, unambiguous, and verifiable. Apply the identifier grammar and family definitions in Section 5.3. Classify its primary family or subtype, write binding behavior with **shall**, assign one Must, Should, Could, or Would priority, assign one lifecycle status, and map it to at least one use case and one test case. A requirement may be marked Done only when its mapped tests pass, verification is complete, traceability is current, and a controlled implementation version is known. Evaluate every Applicable or TBD domain profile that intersects the proposed change and follow Section 5.7 before completing the requirements work.

C.8.5 Use-Case Context

Start from an actor goal and a system boundary. Give each use case its own table, identify participating actors, preconditions, trigger, success outcome, main flow, alternate and exception flows, postconditions, mapped requirements, and mapped tests. Describe externally observable interaction rather than internal implementation detail. Maintain reciprocal mappings to every related requirement.

C.8.6 Architecture Context

Trace each architectural decision to requirements, quality attributes, constraints, risks, and interfaces. Identify alternatives and explain the trade-off that selected the proposed structure. Keep the context, logical, process, development, physical, interface, and implementation views consistent. Use TikZ for diagrams. Store each TikZ source under its owning chapter as required by Section A.8, include it from that chapter, and compile the PDF.

C.8.7 Testing and Evidence Context

Derive tests from requirements and use cases. Define preconditions, inputs, procedure, expected result, and retained evidence. A test definition is not a test result. Verification records objective compliance with requirements; validation records whether stakeholder needs are satisfied in intended use. Never create passing evidence or approval from an expected result.

C.8.8 Release Context

Review open, in-progress, done, and retired requirements; unresolved defects; verification and validation results; operational readiness; migration and rollback provisions; known limitations; and required approvals. Report a gap instead of treating missing evidence as acceptance. Only an authorized human may approve release.

C.8.9 Document-Authoring Context

Follow Chapter A for file naming, LaTeX conventions, diagrams, installation, and builds. Update the glossary and index when introducing a controlled term. Update document history with a link to the changed location. Before creating or renaming a project-owned file, ask for or confirm the project file namespace and apply it consistently in accordance with the document-authoring requirements. External source references, including footnotes, shall use BibTeX citations in accordance with DOC-BIB-001 and Citations and Bibliography. Every TikZ source shall satisfy the path, inclusion, caption, label, and reference requirements in Section A.8 and connector-layout rule DOC-TIKZ-002. Do not edit generated auxiliary files or the PDF directly.

C.9 Mandatory Consistency Rules

1. Preserve every stable ID. Do not reuse an ID belonging to a retired or deleted proposal.
2. Use the scope and family codes in Section 5.3. Before using an undefined scope or family, propose its code, name, boundary, rationale, and examples; check for semantic overlap and identifier collision; obtain authorized-human approval; and update the convention, category catalog, this guide, and the applicable register. Do not rename existing IDs when extending the convention.
3. Use **shall** for binding requirements. Use priority terms only in the Priority field and with the definitions in the glossary.
4. Give every requirement a priority, status, In Version value, use-case mapping, and test-case mapping.
5. Enter ∅ when a requirement is not Done and no completed implementation version applies.
6. Maintain reciprocal mappings among user stories, requirements, use cases, tests, traceability, verification, and validation.
7. Do not copy one controlled statement into several authoritative locations. Link to its stable ID.
8. Retain retired requirements and their history. Record the rationale and replacement when applicable.
9. Use project evidence for factual claims. Label an unsupported suggestion as a proposal and report the evidence needed to decide it.
10. Record each domain profile as Applicable, Not Applicable, or TBD with an authorized decision and rationale. Load every Applicable or relevant TBD profile; do not duplicate project facts in the profile appendix.

11. Before creating or renaming any project-owned file, ask for or confirm the project `<nameSpace>`; do not invent it. Prefix every new project-owned file name with the confirmed namespace.

12. Use TikZ for every diagram. Every TikZ source shall use the directory

    `TikZ/<includingFileBaseName>/`

    and shall be named `<nameSpace><DrawingName>.tex`. The directory shall exactly match the base name of the LaTeX file containing its `\input` command. The including chapter shall place it in a numbered figure, add a descriptive caption and stable label, and reference the figure in its text. Apply the connector-routing and label-clearance rule DOC-TIKZ-002.

13. Apply the linked architecture references rule and DOC-AI-003; check links to authoritative definitions, including each AIS row and its Context, Steps, and Acceptance.

14. Link every document-history entry to the location where the change occurred.

15. Compile after structural, cross-reference, glossary, index, diagram, or table changes and visually inspect the affected pages.

C.10 Skill-Specific Steps and Acceptance

The skill definitions below provide Context, Steps, and Acceptance, not an agent count or a second execution workflow. Table C.2 links each AIS to its context, task steps, and acceptance criteria. Apply the shared execution workflow in Chapter D; an AIS may be assigned to one agent or partitioned among several agents under that workflow. The criteria below concern completion of the assigned task, not human approval of a baseline or release. Also apply the completion checklist.

C.10.1 Estimate AI Work

AI Skill: `AIS-01`.

Steps

Read Planning and Estimation Context. Identify affected artifacts and acceptance criteria, split the work into bounded assignments, and estimate effort and token ranges with assumptions. Compare sequential execution with delegation, including integration and review cost.

Acceptance

The estimate identifies uncertainties, dependencies, ownership, and a justified execution recommendation; any required authorization is obtained before dependent costly work.

C.10.2 Initialize a Project

AI Skill: AIS-02.

Steps

Ask an authorized human for the project file namespace, or confirm an already configured namespace, before creating or renaming files. Do not invent it. Replace `\FileNamespace`, project metadata, and bracketed placeholders only from supplied facts, then apply the namespace to every new project-owned file. Identify available domain profiles and ask an authorized human to confirm each profile's applicability. Record the decisions in the project profile table. Include each Applicable or TBD profile in the initialization context; remove an inapplicable profile appendix only after its decision is recorded. Create a gap list covering purpose, scope, stakeholders, system boundary, interfaces, constraints, quality attributes, risks, evidence, and approvals. Resolve consequential gaps before developing detailed artifacts. Work in dependency order from context and needs through requirements, use cases, architecture, tests, evidence, and release.

Acceptance

Project metadata is supported, profile decisions are recorded, and unresolved facts remain explicit gaps rather than invented content.

C.10.3 Tailor an Embedded System

AI Skill: AIS-03.

Steps

Review the approved profile selection and controlled hardware evidence. Map affected profile concerns to their core artifacts, identify gaps, and select the additional AI Skills needed for requirements, architecture, tests, or evidence.

Acceptance

Applicable concerns are covered or have approved Not Applicable dispositions; project facts remain in their authoritative chapters and hardware assumptions are explicit.

C.10.4 Create Repository AI Bootstrap Files

AI Skill: AIS-04.

Steps

Follow the Repository AI Bootstrap steps, confirm tools and namespace, and create only applicable discovery files pointing to the task guide and shared execution workflow.

Acceptance

Discovery filenames match the confirmed tools; pointers resolve and substantive policy remains in the LaTeX sources.

C.10.5 Develop User Stories

AI Skill: `AIS-05`.

Steps

Read stakeholder evidence and the User-Story Context. Draft beneficiary, goal, value, acceptance criteria, priority, and stable links to requirements. Identify unresolved stakeholder questions.

Acceptance

Each changed story meets the User Stories review rules and has reciprocal traceability; proposed needs remain distinguishable from approved needs.

C.10.6 Add or Change a Requirement

AI Skill: `AIS-06`.

Steps

Identify the originating need and affected use cases. Draft a singular verifiable obligation, select its scope, primary family, priority, and status, and assign a stable ID using Section 5.3. If no defined scope or family fits, complete the proposal and human-approval procedure before assigning the ID. Add or update mapped use cases and tests, then update traceability and verification. Confirm every link in both directions and add a linked history entry.

Acceptance

Each changed requirement satisfies the Requirements Context and has complete reciprocal mappings; unsupported approvals and implementation status are absent.

C.10.7 Add or Change a Use Case

AI Skill: `AIS-07`.

Steps

Identify the actor goal, system boundary, and mapped requirements. Complete one use-case table with success, alternate, and exception behavior. Add mapped tests, update requirement mappings and traceability, and confirm that no flow introduces an unrecorded requirement.

Acceptance

Each changed use case has complete actor flows, outcomes, requirement links, and test links, with no unrecorded obligation.

C.10.8 Develop or Change Architecture

AI Skill: AIS-08.

Steps

Identify the requirements and quality attributes that drive the decision. Describe alternatives, trade-offs, risks, interfaces, and the selected structure. Update every affected architectural view and diagram. Confirm that the proposed design neither omits a driving requirement nor silently adds an unapproved constraint.

Acceptance

The decision is traced to its drivers, alternatives and risks are recorded, and affected views and interfaces agree.

C.10.9 Add Tests or Evidence

AI Skill: AIS-09.

Steps

Derive each test from mapped requirements and use-case flows. Define repeatable steps and objective expected results. Link retained evidence only after an actual execution. Update verification or validation records without converting an expected outcome into a claimed result.

Acceptance

Test definitions are repeatable and linked; expected outcomes are distinguished from observed results and retained evidence.

C.10.10 Perform Verification

AI Skill: AIS-10.

Steps

Identify the requirement baseline, candidate version, mapped tests, and controlled execution evidence. Evaluate objective compliance and update the verification register with actual results, failures, and gaps.

Acceptance

Verification dispositions trace to requirements, candidate versions, and retained evidence; missing or failed evidence is reported without claiming a pass.

C.10.11 Perform Validation

AI Skill: AIS-11.

Steps

Prepare intended-use scenarios from stories and use cases, agree success criteria and participants, and collect actual stakeholder observations and authorized dispositions. Update the validation register and report unmet needs.

Acceptance

Validation records identify the intended context, stakeholder evidence, and human disposition; agent review and passing tests are not substituted for stakeholder validation.

C.10.12 Review and Prepare a Release

AI Skill: AIS-12.

Steps

Audit IDs, mappings, statuses, versions, unresolved assumptions, verification, validation, operational readiness, and release criteria. Report contradictions and missing evidence. Prepare a release recommendation for human approval; do not approve the release.

Acceptance

The recommendation identifies readiness, missing evidence, unresolved issues, and required human approvals without claiming release authorization.

C.10.13 Review Consistency

AI Skill: AIS-13.

Steps

Apply the Mandatory Consistency Rules to affected registers, terminology, statuses, versions, and reciprocal links. Record concrete contradictions with source locations and propose corrections.

Acceptance

Every reviewed inconsistency has a supported disposition or remains an explicit open issue; stable IDs and authoritative locations are preserved.

C.10.14 Compile or Troubleshoot

AI Skill: AIS-14.

Steps

Follow the Document-Authoring Context and Chapter A. Run the complete build, diagnose errors in authored sources, regenerate references and indexes, and render affected pages.

Acceptance

The build succeeds, cross-references resolve, and affected pages have no new clipping or overlap; any remaining build limitation is reported explicitly.

C.11 Example AI Requests and Prompts

Replace bracketed text with project facts. A prompt does not override this guide, the controlled project evidence, or human approval boundaries.

Prompt formatting. Present each copy-and-paste AI request under a descriptive subsection heading as a continuous, unboxed paragraph in typewriter font (`\ttfamily`), matching the original verbatim examples. Let LaTeX wrap lines automatically to the available width. Do not insert manual line breaks or use a verbatim block for prose prompts. Preserve paragraph breaks only when the request contains distinct paragraphs. Do not add a frame, line numbers, or syntax highlighting. Apply this convention to prompt examples throughout the manual, including Appendix D.

C.11.1 Initialize the Architecture Book

```
Read ftdi2AIAgentGuide.tex and use the AIS-02 task steps for [Project Name].
Execute using Appendix D. Review the supplied project evidence and create a
gap list before editing.  Identify missing scope, stakeholders, interfaces,
constraints, quality attributes, risks, evidence, and approvals.  Identify
the available domain profiles and ask me to confirm whether each is
Applicable, Not Applicable, or TBD. Read each Applicable or relevant TBD
profile and add its concerns to the gap list.  After the gap list is
reviewed, replace only supported placeholders and develop the document in
dependency order.  Compile the complete PDF and report unresolved human
decisions.
```

C.11.2 Tailor an Embedded System

```
Read ftdi2AIAgentGuide.tex and ftdi2EmbeddedSystemsProfile.tex. Apply the
Embedded Systems Tailoring Profile to [Project Name]. Ask me to confirm the
project namespace, profile applicability, platform class, hardware
revisions, toolchain, interfaces, constraints, applicable standards, and
available evidence. Add embedded concerns to the gap list before editing.
After human review, place project-specific requirements, architecture,
diagrams, tests, and evidence only in their authoritative core chapters. Do
not invent hardware facts or verification results. Build the complete
document, inspect the log, and visually inspect the affected PDF pages.
```

C.11.3 Create Repository AI Bootstrap Files

```
Read the Repository AI Bootstrap section in ftdi2AIAgentGuide.tex and treat
that LaTeX chapter as authoritative. Confirm which AI tools this repository
uses and the project file namespace. Create only the applicable repository
discovery files, such as AGENTS.md or CLAUDE.md, and the generic SKILL.md
that directs the agent to the AI Skills Register. Keep every bootstrap file
short: point to ftdi2Manual.tex, ftdi2AIAgentGuide.tex, the AI Skills
Register, ftdi2AIImplementationWorkflow.tex, and ftdi2DocumentGuide.tex. Do
not duplicate architecture rules. Verify that each file directs
architecture-related changes back to the authoritative LaTeX sources.
```

C.11.4 Develop Requirements

```
Read ftdi2AIAgentGuide.tex and follow the Requirements context and task
steps for AIS-06. Execute using Appendix D. Load every Applicable or TBD
domain profile that intersects the work. From the approved objectives and
user stories, propose singular, feasible, and verifiable requirements.
Classify each one, use shall for the binding obligation, assign one priority
and one status, and map it to at least one use case and test case. Do not
complete requirements work until each applicable profile concern maps to a
new or existing requirement, or to an approved Not Applicable decision with
rationale. Do not mark work Done or assign an implementation version
without evidence. Update reciprocal mappings, history, and the index; then
build and inspect the affected pages.
```

C.11.5 Develop Use Cases

```
Read ftdi2AIAgentGuide.tex and follow the Use-Case context and task steps
for AIS-07. Execute using Appendix D. Derive use cases from the approved
actor goals and mapped requirements. Give each use case its own table with
actors, trigger, preconditions, main flow, alternate and exception flows,
outcome, postconditions, requirements, and tests. Maintain reciprocal links
```

```
and report any flow that requires a new requirement.  Build and inspect the
affected pages.
```

C.11.6 Develop an Architecture Decision

```
Read ftdi2AIAgentGuide.tex and follow the Architecture context and task
steps for AIS-08.  Execute using Appendix D for [decision or subsystem].
Identify driving requirements, quality attributes, constraints, interfaces,
and risks.  Compare reasonable alternatives and document the trade-off
supporting the proposed choice.  Update every affected architecture view and
editable diagram source.  Do not present the proposal as approved.  Compile
the document and inspect all affected diagrams and pages.
```

C.11.7 Develop Tests and Traceability

```
Read ftdi2AIAgentGuide.tex and follow the Testing and Evidence context and
AIS-09 task steps.  Execute using Appendix D. For [requirements or use
cases], define repeatable tests with inputs, preconditions, steps, expected
results, and required evidence.  Update all reciprocal mappings and the
traceability matrix.  Do not invent an execution result or mark verification
complete.  Compile the document, check references, and inspect the affected
tables.
```

C.11.8 Review Release Readiness

```
Read ftdi2AIAgentGuide.tex and follow the Release context and task steps for
AIS-12 and [release].  Execute using Appendix D. Audit requirement status
and implementation versions, traceability, verification, validation,
defects, operations, rollback, known limitations, and approvals.  Separate
confirmed evidence from gaps and assumptions.  Prepare a release-readiness
recommendation for human review, but do not approve release.  Build and
inspect the final document.
```

C.12 Human Decisions and Information Boundaries

An authorized human shall approve scope, requirements, architecture decisions, security and privacy posture, safety decisions, evidence sufficiency, waivers, validation conclusions, and release. The agent shall present alternatives and evidence without claiming that review has occurred.

Do not provide an AI agent with credentials, secrets, personal data, production data, export-controlled material, or other information it is not authorized to process. Before using external sources, confirm that the task permits network access and that the source may be incorporated. Record material prompts, assumptions, corrections, and unresolved issues in Chapter E.

C.13 Completion Checklist

Table C.3: AI-Assisted Change Completion Checklist

Area	Required Completion Evidence
Scope	The requested outcome is complete; unrelated chapters and user changes remain untouched.
Domain profiles	Every available profile has an authorized applicability decision and rationale; every Applicable or relevant TBD profile was included in the task context, and project facts remain in core chapters.
Evidence	New factual statements are supported by supplied or controlled evidence; proposals, assumptions, and unknowns are identified.
Terminology	Changed content uses the established AI Skill names and `aia` role identifiers under Consistent terminology and DOC-AI-004.
Language	Binding requirements use shall; priority and status use the controlled terms and definitions.
Identifiers	Stable IDs follow the documented scope and family convention, remain unique, and are neither renumbered nor reused; every new scope or family has its definition and authorized-human approval recorded before use.
File names	The project namespace was supplied or explicitly confirmed; every newly created project-owned file begins with it, and no namespace was invented.
AI bootstrap	Every applicable repository AI discovery file is a thin pointer to the authoritative LaTeX architecture; tool-mandated filenames are documented exceptions, and no architecture rules are duplicated in Markdown.
Traceability	Every changed requirement, use case, test, verification record, and validation record has complete reciprocal mappings.
Reference links	References to defined architecture artifacts reach their authoritative definitions; AIS references reach individual register rows, and each row and skill definition provide reciprocal navigation under DOC-AI-003.
History	The newest approved document change appears at the top of document history and links to the changed location.
Glossary and index	New controlled terms and important concepts are defined and indexed where applicable.
Diagram source	Every diagram uses TikZ; each TikZ directory exactly matches its including LaTeX file's base name, each drawing file begins with the project namespace, and the drawing is included, captioned, labeled, and referenced.
Connector layout	Inspect every drawing against DOC-TIKZ-002.
Build	The complete platform build succeeds; citations, glossary, index, diagrams, and cross-references are regenerated.
Log review	No fatal error, undefined reference, duplicate label, missing file, or unresolved layout warning remains.

AI-Assisted Change Completion Checklist (continued)

Area	**Required Completion Evidence**
Visual review	Affected PDF pages show readable tables, correct page breaks, current diagrams, working headings, and no clipped or overlapping content.
Human review	Decisions requiring authority are clearly identified and are not represented as approved without evidence.

Appendix D

AI-Assisted Execution Workflow

D.1 Chapter Responsibility

Organize AI-assisted work from planning through integration and human review.
Expected result: Coordinated execution with explicit checks and approval boundaries.

D.2 Purpose and Scope

Use this shared workflow for the AI skills selected in Chapter C: project initialization, architecture documents, requirements, user stories, analyses, code, tests, verification, validation, and release-readiness recommendations. Appendix C defines the task inputs, context, steps, and acceptance criteria; this appendix defines how to plan, assign, execute, integrate, check, and review the work with one or more agents. Agent count does not change a task's acceptance criteria or human approval boundaries. Roles describe responsibilities, not a required agent count. Codex supplies the concrete examples; see Other AI Tools for adapting the workflow.

Follow the source-of-truth and human-decision rules in Appendix C. Use approved baselines when implementing code; document drafts and analyses may propose changes but shall not claim human approval. Apply checks appropriate to the artifact: document builds and reciprocal mappings for architecture content, project builds and tests for code, controlled evidence for verification, and stakeholder observations for validation. Project-specific commands and completion criteria belong in the implementation and development-plan chapters; controlled relationships remain in their authoritative registers.

Figure D.2 connects Chapter C to this workflow. Appendix C supplies the selected skill definitions; this workflow applies them with one or multiple agents and checks the results against their Acceptance. The final check uses the completion checklist; completion does not imply human approval. This overview summarizes the successful path. Use Figure D.1 and the correction and escalation rules for failed checks and human decisions.

D.3 Starting the Workflow in an AI Chat

Open a Codex chat in the project workspace and replace the bracketed fields. Starting delegated work involves three distinct responsibilities:

1. **Codex supplies the tools.** The application exposes tools to create, message, wait for, and interrupt subagents. This appendix and its prompts do not enable those tools. Current local Codex releases enable subagents by default; if disabled, enable `agents.enabled` in the Codex configuration before requesting delegated work.[1]

2. **The human authorizes delegation.** The request or applicable project instructions permit subagents and set scope, budget, concurrency, and decision boundaries. Tool availability alone is not authorization.

3. **The aiaCoordinator decides and invokes.** Confirm tool availability and authorization, estimate the work, and choose one agent or independent bounded assignments. If delegation is useful and permitted, invoke the actual Codex subagent tools and wait for their results before integration.

Use the starting request in the project chat; creating an API session or running Python is not required. Naming AI agent roles in a prompt does not launch subagents. Name authoritative sources and deliverables; the aiaCoordinator reports actual agents used and project-specific commands. See Choose and Invoke Agents for assignment and invocation requirements. The

[1] Codex subagent configuration and delegation guidance: [7].

examples follow the prompt formatting convention in Appendix C.

D.3.1 Start Any Selected AI Skill

```
In [project workspace], complete [outcome] using AIS-[IDs] from Table C.2 in
ftdi2AIAgentGuide.tex. Read their sources, Context, Steps, and Acceptance.
Follow the shared execution workflow in ftdi2AIImplementationWorkflow.tex.
Act as aiaCoordinator. For substantial work, first apply AIS-01: estimate
effort and token ranges, assumptions, and dependencies. You are authorized
to use Codex subagent tools for independent bounded tasks, after confirming
their availability, within project instructions and a maximum of [N]
concurrent agents including yourself. Explain the delegation decision and
assign each agent its AI Skill, Context, Steps, output, Acceptance,
dependencies, and file ownership. Use one agent when delegation adds little
value; if subagents are unavailable, report that and work sequentially.
Integrate outputs, run the applicable checks, and obtain independent review
when warranted. Arrange stakeholder validation where needed. After two
unsuccessful corrections of the same failure, pause dependent work and
present evidence and options. Ask me to resolve material decisions or
required cost approvals before dependent work. Report actual agents used,
changes, commands, evidence, and gaps. Do not submit, approve a baseline,
or authorize release for me.
```

D.3.2 Develop an Approved Change

```
In [project workspace], develop [approved change] against [approved
architecture and requirements]. Read the AI agent guide and Appendix D. As
aiaCoordinator, list components, criteria, checks, and decision gaps.
Assign bounded tasks with file ownership. You are authorized to use Codex
subagent tools for independent work, with at most [N] concurrent agents
including yourself. Confirm tool availability, explain the delegation
decision, and invoke parallel aiaDeveloper agents only when useful;
otherwise work sequentially. Integrate, verify, and obtain independent
review. Arrange stakeholder validation; record only observed results.
After two failed corrections of one check, pause dependent work and show
evidence and options to revise or stop. Ask me to decide material scope,
architecture, safety, security, release, or baseline issues. Report files,
commands, checks, trace links, validation evidence, assumptions, and gaps.
Do not submit or approve the change for me.
```

The same sequence can prepare a requirements package before implementation. For that use, aiaDeveloper agents produce bounded analyses and drafts, the aiaCoordinator is the single writer for shared controlled registers, and checks include traceability and document

builds. Replace its placeholders with project facts.

D.3.3 Prepare a Requirements Package

```
In [project workspace], develop a reviewable requirements package for
[approved scenario]. Read the approved architecture, stakeholder scenario,
applicable domain profiles, existing registers, AI agent guide, and Appendix
D. List missing source facts as questions; do not invent project facts. Act
as aiaCoordinator. First show the sources, artifacts, acceptance criteria,
and task boundaries. You are authorized to use Codex subagent tools, with
at most four concurrent agents including yourself. Confirm tool
availability and explain the delegation decision. If independent work
warrants it, invoke up to three AI agents: aiaDeveloperA for stories and
validation scenarios, aiaDeveloperB for functional, interface, and fault
requirements, aiaDeveloperC for verification and test design. Keep one
writer per shared file; if subagents are unavailable, work sequentially.
Integrate the outputs, create reciprocal trace links, run document and build
checks, and report actual results. Obtain an independent review of the
integrated candidate. After two unsuccessful corrections of a failed check,
pause dependent work and show me evidence and options. Ask me to decide
material scope, architecture, safety, or baseline issues. Report changed
files, trace gaps, assumptions, evidence, and open issues. Do not submit or
approve a baseline for me.
```

D.3.4 Other AI Tools

Codex is the concrete application used in the starting examples. The AI Skills, Context, Steps, Acceptance, and responsibilities of aiaCoordinator, aiaDeveloper, and aiaReviewer also apply when using other AI tools. The application provides the execution capabilities; a model name alone does not establish access to project files or tools for delegating work.

Claude Code. Supports subagents with separate context and specified instructions, tools, and permissions. Adapt the starting request to Claude Code's available subagents and delegation tools; confirm the instructions and sources each subagent receives.[2]

Gemini. Gemini CLI supports subagents exposed to the main agent as tools; a delegated subagent returns its findings to the main agent. Confirm which subagents and permissions are available in the installed application before assigning work. Capabilities of Gemini CLI shall not be assumed for every Gemini chat interface.[3]

Grok. Grok offers a multi-agent research feature that coordinates agents for research and synthesizes their findings. Before using it for this workflow, confirm whether the selected Grok application or API supports the required project access, assignment

[2] Claude Code subagent documentation: [8].
[3] Gemini CLI subagent documentation: [9].

boundaries, and checks. Research-agent collaboration alone does not demonstrate that separate aiaDeveloper agents edited and verified project artifacts.[4]

For any other application, replace "Codex subagent tools" in the example request with its confirmed delegation capability. Keep the same AIS IDs, Context, Steps, Acceptance, role names, human authorization, file ownership, concurrency limits, verification, validation, and escalation rules. Identify the application and version in the execution report and record actual agents and checks used. If delegation is unavailable, report that limitation and complete the applicable work with one agent. A prompt cannot enable missing tools, and role names alone do not launch separate agents.

D.4 Plan the Selected Tasks

Before changing artifacts, select the applicable AI Skills and identify the requested outcome, approved constraints, relevant domain profiles, task acceptance criteria, affected sources, and required checks. For code changes, identify the approved requirements, architecture, interfaces, and tests. Use the AI Skills Register in Chapter C to load the affected sources and reciprocal mappings. Record any gap or conflict as a question with its implementation impact. An authorized human resolves a material architecture, scope, safety, security, or release decision before work that depends on that decision proceeds. An agent may prepare independent analysis while the decision is pending; it shall not silently choose an answer.

Apply AI skill `AIS-01` in the AI Skills Register before substantial or delegated work. Break the approved work into bounded tasks with inputs, expected outputs, acceptance criteria, dependencies, and ownership of files or components. The aiaCoordinator decides whether delegation will shorten the work or improve review coverage enough to justify its coordination and token cost. Use one agent for short tasks, ordered chains of dependent steps, or changes concentrated in the same mutable files. Delegate in parallel only when tasks can make progress independently. Give each aiaDeveloper a distinct edit boundary or an isolated workspace; read-only investigations may share the same sources.

D.5 Choose and Invoke Agents

No AIS inherently requires multiple agents. Use one agent for small edits, a dependent chain of steps, or work concentrated in shared mutable registers. Several AI Skills can be completed sequentially by one agent. Conversely, one AIS can be partitioned among multiple aiaDeveloper agents when its outputs are independent. Examples include separate architecture-view analyses, tests for independent components, or separate requirements-family drafts. Dependent outputs wait for their inputs; parallelism does not remove that dependency.

The human sets the outcome, authorization, budget, concurrency limit, and project constraints. Within those limits, the aiaCoordinator decides whether to delegate, after applying `AIS-01` when required. Explain the decision, expected benefit, and coordination cost. Obtain

[4] Grok multi-agent research documentation: [10].

human authorization when project instructions require it, when delegation is not permitted, or when a proposed plan exceeds agreed scope or cost. Do not interpret an estimate as approval. If no delegation is authorized, perform the work with one agent.

To invoke multiple agents, the aiaCoordinator shall:

1. Confirm that the selected application exposes subagent tools and that the human request or applicable project instructions authorize their use, as described in Starting the Workflow in an AI Chat. Report its absence and use the single-agent path when delegation is unavailable. For applications other than Codex, apply Other AI Tools.

2. Create one bounded assignment per aiaDeveloper with applicable AIS IDs, authoritative source paths, linked Context, Steps, Acceptance, expected output, dependencies, and a distinct edit boundary. Reading shared sources is allowed; controlled shared registers have one writer.

3. Launch assignments through the selected application's actual subagent tools, within the total concurrency limit. Use the available messaging, waiting, and interruption tools to coordinate their execution. A role name written in a prompt is not evidence that a separate agent was launched.

4. Monitor returned results, resolve conflicts, and integrate ready outputs. Record actual agents used, assignments, checks, and unresolved issues. Do not claim parallel execution if the work was performed sequentially.

For example, the aiaCoordinator may delegate story and validation-scenario analysis under `AIS-05` and `AIS-11`, requirements drafting under `AIS-06`, and test design under `AIS-09`. The aiaCoordinator integrates their drafts into shared registers after dependencies are resolved. These tasks may use several AI Skills; they do not create new skill IDs.

D.6 Responsibilities and Handoffs

The prefix `aia` means AI agent. These role identifiers follow the terminology convention.

aiaCoordinator
: Confirms scope and decisions, partitions independent work, assigns boundaries and acceptance criteria, reconciles results, integrates the change, runs system-level checks, and reports evidence and open issues.

aiaDeveloper
: Produces the assigned analysis, document draft, code, tests, or evidence, changes only the assigned area, preserves the approved architecture and project conventions, runs relevant local checks, and returns a summary of changed files, results, assumptions, and unresolved issues.

aiaReviewer
: Examines the integrated result against requirements, architecture, interfaces, tests, and evidence. The aiaReviewer reports concrete findings and does not mark a decision or release approved on behalf of the human owner.

Use the role names above consistently in instructions, figures, and reports. The aiaDeveloper is the agent assigned to a bounded task, including document authoring or analysis; the number of aiaDeveloper agents does not change the role name. One agent may coordinate and develop a small change, then perform a separate self-review pass. Report that pass as self-review, not independent review. When independent review is required, use an uninvolved agent or human; if none is available, report the review as pending. For a delegated change, the aiaReviewer should be independent of its implementation and should review only after integration produces a stable candidate. The aiaCoordinator resolves conflicting aiaDeveloper results and repeats checks after corrections. Figure D.1 shows the optional parallel path and the later review handoff.

D.7 Develop, Integrate, and Verify

Each aiaDeveloper keeps changes traceable to the approved task and supplies local check results. The aiaCoordinator inspects and integrates all outputs, checks shared interfaces and architecture constraints, updates the affected documentation and reciprocal traceability, and runs the applicable complete build and checks: document compilation, reference and mapping checks for documents; project builds and relevant tests for code; evidence checks for verification and validation records. A failed check returns to the responsible task for correction, followed by integration and verification again. Limit this cycle to two unsuccessful correction attempts for the same failure. If the check still fails, or the cause requires a decision outside the approved scope, the aiaCoordinator reports the evidence and options to the human owner and pauses dependent work until the owner decides whether to revise scope, continue with a new bounded plan, or stop. A revised plan returns to task partitioning and the decision about whether parallel delegation is useful; a stop ends the workflow without review of an unverified candidate. Record actual commands, results, controlled artifact or implementation version, and evidence locations; do not infer a pass from a proposed test or an agent's assertion.

The review checks the integrated candidate for requirement coverage, architecture conformance, unintended changes, missing tests, and unsupported evidence. The aiaCoordinator addresses findings and obtains another review when corrections affect the reviewed behavior. Human owners retain the authority for the decisions named in Chapter C. A verified and reviewed candidate proceeds to stakeholder validation when the task requires validation, with open issues reported. A document-only task may finish after its acceptance criteria, checks, and required review are met; it shall not claim implementation verification or stakeholder validation from a successful document build.

D.8 Validate with Stakeholders

When the selected task requires validation, prepare realistic scenarios mapped to the approved user stories and use cases. Stakeholders evaluate the candidate in its intended context against agreed success criteria. The aiaCoordinator may prepare the scenarios and organize evidence, but records validation as passed only from actual stakeholder observations and an

authorized human disposition. Update the authoritative validation register with the result, evidence location, unmet needs, and requested changes. Passing implementation tests or agent review alone does not establish validation.

If validation reveals a gap, the aiaCoordinator reports its impact and options to the human owner. A material change to scope, requirements, or architecture returns to planning and the delegation decision. A correction within approved scope returns to bounded development; repeat affected integration, verification, independent review, and validation before reporting completion. The authorized human retains acceptance and release decisions.

D.9 Project Tailoring Checklist

When instantiating this template, record the following in the project-owned implementation and development-plan chapters rather than in tool bootstrap files:

- The AI tools permitted, how aiaDeveloper agents are launched, the maximum useful concurrency, and whether isolated workspaces are required.
- The project file or component ownership rules, integration owner, and required code review or change-control process.
- The exact build, static analysis, unit, integration, and system test commands, including any hardware-in-the-loop or manual checks.
- The required traceability, test evidence, implementation-version record, and acceptance criteria for completion.
- The validation scenarios, stakeholder participants, success criteria, evidence locations, and authorized owner of the validation disposition.
- The human decision owners and the points at which unresolved decisions pause dependent implementation or release.

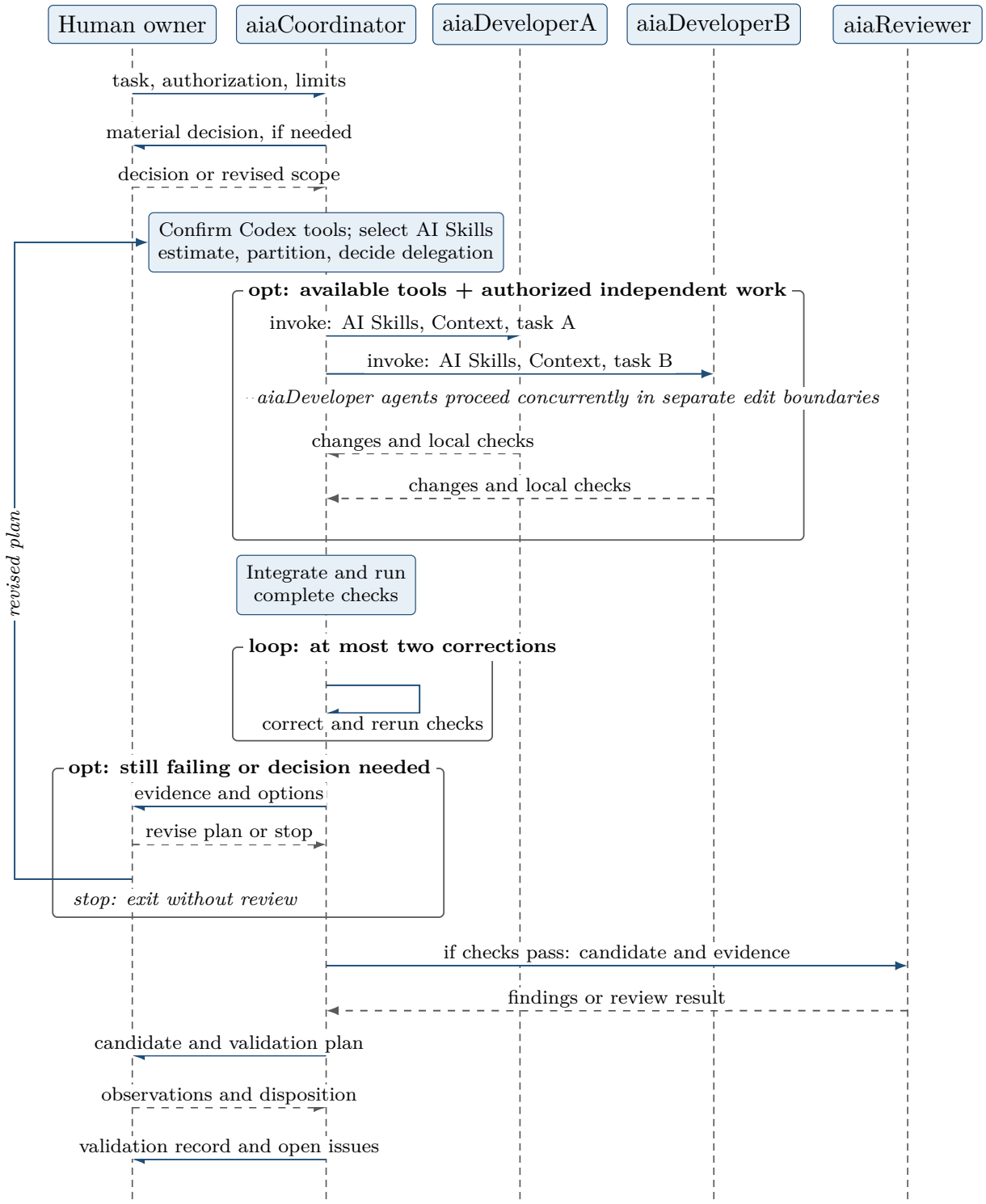


Figure D.1: AI-assisted development and validation handoffs. Codex supplies subagent tools; the human authorizes delegation; the aiaCoordinator decides and invokes. Parallel aiaDeveloper agents are used only for independent tasks; otherwise the aiaCoordinator develops tasks sequentially.

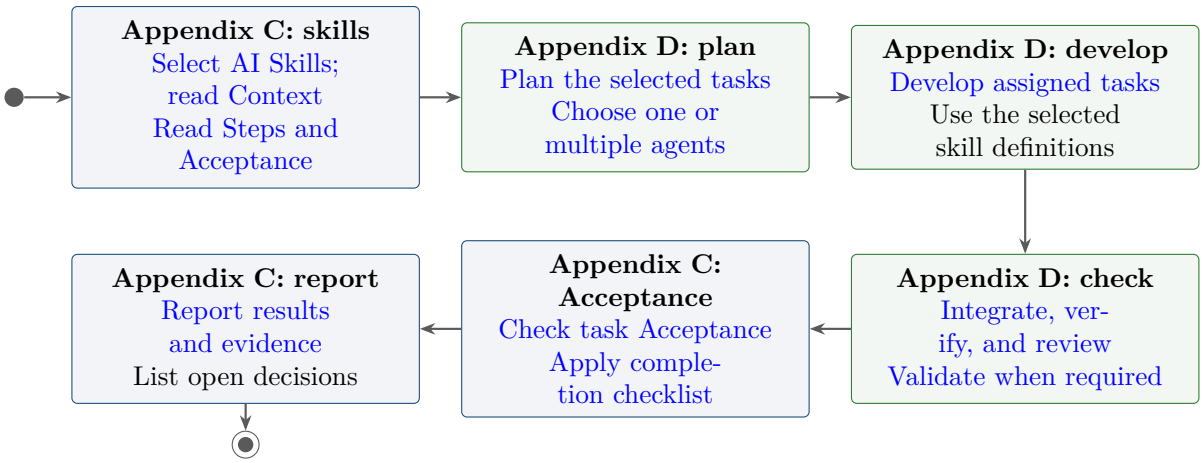


Figure D.2: Activity connecting Appendix C skill definitions and Acceptance to Appendix D execution. The agent-count decision belongs to Appendix D for every selected task.

Appendix E

AI Prompt Log

E.1 Chapter Responsibility

Preserve the provenance of material AI assistance and its human review.
Expected result: An auditable account of how AI contributed to the engineering record.

E.2 How to Use This Appendix

Use this log when AI assistance materially affects requirements, architecture, code, tests, or release evidence. Record enough context for a reviewer to understand the input, output, assumptions, and human corrections.

E.3 Initialization Interaction

Date: November 12, 2026. Tool: Codex. Role: aiaCoordinator, with drafting and self-review performed by the same agent. The user requested AIS-02 initialization from an unchanged template copy, a facts/gaps/files/checks preview, confirmation before dependent edits, separate TikZ sources, complete PDF build and inspection, and a later acceptance/publication gate. The user then confirmed the project name, namespace, owner and approver, documentation-only scope, Applicable Embedded Systems profile, and user ownership of unknown technical details. See confirmed decisions and owned questions. No product facts or evidence were supplied. The implementation uses no delegated agents. At that stage, owner acceptance, independent review, publication authorization, and repository confirmation were pending. Subsequent owner disposition is recorded in publication authorization. Technical examples from the template were not adopted as project artifacts.

E.4 Owner Identity Correction

On November 12, 2026, Nkeh P. Boh supplied their name, organization Stevens Institute of Technology, and email nboh@stevens.edu, and requested replacement of the generic owner

fields. Codex updated the shared metadata, front matter, PDF author, ownership references, and Q-01. The change uses local branch `codex/update-owner-details`; publication and its pull request remain pending the owner's separate publication authorization at that time. Technical questions and product release decisions remain open; subsequent project acceptance is recorded below.

E.5 Publication Authorization

On November 12, 2026, Nkeh P. Boh stated: "I accept the initialized project and authorize publication." The owner directed a source commit on main and push to the repository identified in the publication record, with branches and pull requests for every later change. Codex records this actual human disposition and prepares the initial publication. It does not infer technical approvals or product release from this authorization.

E.6 Requirement Scope or Family Extension Record

When an AI-assisted change proposes a requirement scope or family that is not already defined, record the proposed uppercase code, full name, decision boundary, rationale, representative examples, semantic-overlap review, identifier-collision check, affected register and category-catalog changes, and the authorized human's approval or rejection. Do not assign an identifier from the proposed scope or family before approval.

Appendix F

Test Evidence

F.1 Chapter Responsibility

Retain detailed evidence supporting test and verification conclusions.
Expected result: Accessible evidence behind the results summarized in the main chapters.

F.2 How to Use This Appendix

Store or reference detailed test and verification evidence here when it would make the main chapters difficult to read. Evidence must identify the artifact, result, environment, date, and product revision.

F.3 Project Status

No product evidence has been supplied. Document-build evidence is recorded separately. See Q-10, owned by Nkeh P. Boh.

Appendix G

Document History

G.1 Chapter Responsibility

Distinguish template provenance, authorized initialization, and baseline acceptance. **Expected result:** Traceable changes without unsupported approval claims.

G.2 Project History

November 12, 2026 / 0.3 — Nkeh P. Boh accepted the initialized project and authorized its initial source publication on main. See acceptance and publication, subsequent change control, and authorization provenance. Technical questions remain open.

November 12, 2026 / 0.2 — Owner-authorized correction of name, organization, and email. See project metadata and correction provenance. Owner acceptance was pending at this revision.

November 12, 2026 / 0.1 — Initialization draft prepared by Codex under the Nkeh P. Boh's authorization. Project metadata, scope, and profile selection were confirmed; owner acceptance was pending at this revision. See confirmed decisions, questions, and interaction provenance.

G.3 Template Provenance

Derived from ArchitectureDesignTemplate-2.33 without modifying that source directory. Template version 2.33 is provenance, not the project document version. The upstream history is retained in the preserved template; it does not assert project approvals. Attribution remains in Template Attribution and Chapter A. The source-file hash manifest is `ftdi2TemplateManifest.json`.

Glossary

acceptance criterion An observable condition used to decide whether a need, story, or requirement has been satisfied. 86

Could A desirable capability included only when time and resources permit. 2, 61, 86

Must A capability required in the current release; the release is unacceptable without it. 2, 61, 86

requirement A necessary, singular, unambiguous, feasible, and verifiable statement of capability or constraint. 86

Should An important capability expected in the current release unless a documented constraint prevents it. 2, 61, 86

stakeholder A person, group, or organization that affects the system or is affected by it. 86

traceability The ability to follow an artifact backward to its source and forward to its implementation and evidence. 86

use case A goal-oriented interaction between an actor and the system, including normal and alternate flows. 86

V-model A lifecycle model that pairs each decomposition and specification activity with a corresponding integration and evidence activity. 86

validation Confirmation that the completed system satisfies stakeholder needs in its intended context; building the right system. 86

verification Confirmation that specified requirements have been fulfilled; building the system right. 86

Would A capability assigned to a later implementation version after higher-priority work. 2, 61, 86

Acronyms

AI artificial intelligence. 1, 86

UI user interface. 86

UML Unified Modeling Language. 86

V&V verification and validation. 86

Bibliography

[1] D. Muresan, "ArchitectureDesignTemplate-2.33," user-supplied local architecture template, version 2.33; reviewed November 12, 2026. Developed with OpenAI Codex and Anthropic Claude Code.

[2] *ISO/IEC/IEEE 15288:2023, Systems and Software Engineering—System Life Cycle Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 15 288:2023, 2023. [Online]. Available: https://www.iso.org/standard/81702.html

[3] *ISO/IEC/IEEE 12207:2026, Systems and Software Engineering—Software Life Cycle Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 12 207:2026, 2026. [Online]. Available: https://www.iso.org/standard/90219.html

[4] *Homebrew Installation*, Homebrew, accessed: October 13, 2026. [Online]. Available: https://brew.sh/

[5] *MacTeX Distribution*, TeX Users Group, accessed: October 13, 2026. [Online]. Available: https://tug.org/mactex/

[6] *Installing MiKTeX*, MiKTeX, accessed: October 13, 2026. [Online]. Available: https://miktex.org/howto/install-miktex

[7] *Codex Subagents*, OpenAI, accessed: October 13, 2026. [Online]. Available: https://learn.chatgpt.com/docs/agent-configuration/subagents

[8] *Claude Code: Create Custom Subagents*, Anthropic, accessed: October 13, 2026. [Online]. Available: https://code.claude.com/docs/en/sub-agents

[9] *Gemini CLI: Subagents*, Google, accessed: October 13, 2026. [Online]. Available: https://geminicli.com/docs/core/subagents/

[10] *Grok: Multi-agent Research*, xAI, accessed: October 13, 2026. [Online]. Available: https://docs.x.ai/developers/model-capabilities/text/multi-agent

Index

acceptance criterion, 10
AGENTS.md, 54
AI
    architecture as maintained input, 2
AI agent, 52
    agent-count decision, 77
    aiaCoordinator, 78
    aiaDeveloper, 78
    aiaReviewer, 78
    architecture workflow, 4, 33
    completion checklist, 71
    consistency rules, 62
    context pack, 60
    context routing, 52
    embedded systems tailoring, 49
    example prompts, 52, 68
    execution workflow, 73
    guide activity, 56
    human review, 52, 70
    independent review, 73
    LaTeX installation prompt, 33
    linked references, 53
    parallel delegation, 73
    project initialization, 3
    quick start, 53
    repository bootstrap, 54
    skill selection, 56
    task steps, 63
    terminology consistency, 53
    TikZ requirement, 52
Appendix
    AI Agent Usage Guide, 52
    Document Authoring Guide, 33
architecture
    AI-assisted design, 2

build tools, 33

Chapter
    Introduction, 1
CLAUDE.md, 54

diagram
    TikZ source, 37
document conventions, 33
domain tailoring profile, 4
    embedded systems, 43

embedded systems, 43

hardware–software co-design, 43

introduction, 1

LaTeX, 33

Muresan, Darian, iii

normative language, 1

priority, 1, 11

requirement, 11
    category catalog, 14
    domain profile, 14
    family code, 11
    functional, 13
    identifier convention, 11
    implementation version, 11
    quality, 13
    scope qualifier, 11
    status, 13
requirements traceability matrix, 29

SKILL.md, 54
source of truth, 1, 52
status
    Done, 13

In Progress, 13
Open, 13
Retired, 13

template
project initialization, 3
tailoring, 4
template attribution, iii
template provenance, 33
TikZ, 33
traceability, 29

user story, 10

V-model, 1
validation, 31
validation scenario, 31
verification, 30
verification method, 30